ỦY BAN NHÂN DÂN TP HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

TÌM HIỂU VỀ KOTLIN

MÔN CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

Giảng viên hướng dẫn : Phạm Thi Vương

Nhóm 35 bao gồm các thành viên :

SV1:

SV2: 123

SV3:

THÀNH PHỐ HỒ CHÍ MINH, tháng 4 năm 2026

LỜI MỞ ĐẦU

Trong bối cảnh công nghệ thông tin phát triển không ngừng, các ứng dụng di động ngày nay không chỉ đòi hỏi tính năng đa dạng mà còn yêu cầu sự ổn định cao, tốc độ xử lý tối ưu và giao diện người dùng hiện đại. Java, dù là một ngôn ngữ lâu đời và mạnh mẽ, vẫn bộc lộ những hạn chế về sự rườm rà trong cú pháp và những rủi ro liên quan đến lỗi tham chiếu null (Null Pointer Exception). Chính vì vậy, sự ra đời của Kotlin đã đánh dấu một bước tiến quan trọng, mang đến một ngôn ngữ lập trình hiện đại, an toàn và ngắn gọn hơn. Với việc được Google công bố là ngôn ngữ ưu tiên hàng đầu (Kotlin-first) cho phát triển Android, Kotlin đã nhanh chóng khẳng định vị thế và trở thành một giải pháp toàn diện cho việc xây dựng các hệ thống phần mềm từ di động đến backend. Việc lựa chọn nghiên cứu và ứng dụng Kotlin trở thành một hướng đi chiến lược, giúp người học tiếp cận với tư duy lập trình mới, nâng cao hiệu suất làm việc và chuẩn bị nền tảng vững chắc cho con đường nghề nghiệp trong tương lai.

Mục tiêu của báo cáo này là nghiên cứu một cách hệ thống về Kotlin, từ lịch sử hình thành, quá trình phát triển cho đến những đặc điểm cú pháp nổi bật và các tính năng an toàn vượt trội. Bên cạnh việc tìm hiểu lý thuyết, báo cáo còn hướng tới việc áp dụng kiến thức đã học để xây dựng một ứng dụng thực tế – "Ứng dụng bán sách". Qua đồ án này, nhóm không chỉ củng cố kỹ năng lập trình cốt lõi mà còn hình thành tư duy thiết kế giao diện khai báo với Jetpack Compose và quản lý phụ thuộc khoa học với thư viện Hilt theo chuẩn mực phát triển hiện đại.

Phạm vi của báo cáo tập trung chủ yếu vào việc trình bày các kiến thức nền tảng của ngôn ngữ Kotlin và các bước thực hành liên quan đến bộ công cụ Jetpack Compose. Những công nghệ hỗ trợ như Hilt (Dependency Injection) hay Android Studio sẽ được đề cập trong phạm vi hỗ trợ nhằm làm rõ quy trình triển khai ứng dụng, nhưng không đi quá sâu vào các chi tiết kỹ thuật nằm ngoài phạm vi cốt lõi của ngôn ngữ.

Toàn bộ báo cáo được triển khai theo một cấu trúc logic và mạch lạc. Phần đầu tiên sẽ giới thiệu tổng quan về Kotlin, từ bối cảnh ra đời cho đến vị thế hiện tại trên thị trường công nghệ. Phần tiếp theo đi sâu vào các kiến thức cốt lõi như Null Safety, Data Class, kết hợp với thực hành xây dựng giao diện bằng Jetpack Compose và quản lý đối tượng bằng Hilt, giúp người đọc nắm vững lý thuyết thông qua các ví dụ minh họa trực quan.

Phần ba tập trung mô tả quá trình xây dựng đồ án ứng dụng bán sách, nơi các thành phần kiến thức được kết nối để tạo thành một sản phẩm hoàn chỉnh. Cuối cùng, báo cáo khép lại bằng phần tổng kết, đánh giá những kết quả đạt được, nêu bật các ưu điểm, hạn chế và định hướng phát triển mở rộng cho ứng dụng trong tương lai.

PHẦN 1: TỔNG QUAN VỀ KOTLIN

CHƯƠNG 1: GIỚI THIỆU CHUNG

1. Lịch sử hình thành và phát triển

1.1. Ai/Tổ chức nào đã tạo ra?

Ngôn ngữ lập trình Kotlin được nghiên cứu và phát triển bởi JetBrains – một công ty công nghệ nổi tiếng có trụ sở chính tại Cộng hòa Séc (đây cũng là đơn vị tạo ra những công cụ lập trình phổ biến như IntelliJ IDEA, WebStorm, PyCharm).

JETBRAINS

1.2. Thời điểm ra đời

- Tháng 7 năm 2011: JetBrains lần đầu tiên công bố dự án Kotlin sau một thời gian dài nghiên cứu và phát triển nội bộ.
- Tháng 2 năm 2012: JetBrains chính thức mở mã nguồn dự án Kotlin theo giấy phép Apache 2.
- Ngày 15 tháng 2 năm 2016: Phiên bản Kotlin 1.0 chính thức được phát hành. Đây là cột mốc quan trọng đánh dấu sự ổn định đầu tiên của ngôn ngữ này, cam kết khả năng tương thích ngược lâu dài.
- Tháng 5 năm 2017: Tại sự kiện Google I/O, Google chính thức công bố Kotlin là ngôn ngữ được hỗ trợ ưu tiên để phát triển ứng dụng Android, tạo nên sự bùng nổ về mức độ phổ biến của ngôn ngữ này trên toàn cầu.

1.3. Bối cảnh và lý do ra đời

- Sự hạn chế của ngôn ngữ Java : Vào thời điểm Kotlin bắt đầu được nghiên cứu (khoảng năm 2010-2011), Java là ngôn ngữ thống trị trên nền tảng JVM và Android. Tuy nhiên, Java tồn tại nhiều vấn đề gây khó khăn cho lập trình viên. Lập trình viên phải viết quá nhiều mã nguồn lặp đi lặp lại chỉ để thực hiện những tác vụ đơn giản (như tạo Getter, Setter, Constructors). Java không có cơ chế quản lý các giá trị null một cách chặt chẽ ở mức biên dịch, dẫn đến việc ứng dụng thường xuyên bị lỗi (crash) khi thực thi nếu lập trình viên quên kiểm tra null.
- Nhu cầu thực tế của JetBrains: Là một công ty chuyên sản xuất các công cụ lập trình (IDE), JetBrains sở hữu hàng triệu dòng code bằng Java. Họ nhận thấy rằng việc sử dụng Java làm giảm năng suất làm việc của đội ngũ kỹ sư. Họ cần một ngôn ngữ mới có khả năng:

- Tương thích hoàn toàn với Java (Interoperability): Có thể chạy chung với mã Java cũ trong cùng một dự án mà không cần viết lại toàn bộ.
- Tối ưu hóa công cụ: Ngôn ngữ phải được thiết kế để các IDE có thể phân tích và hỗ trợ lập trình viên tốt nhất (như tự động gợi ý code, phát hiện lỗi sớm).
- Sự bùng nổ của phát triển ứng dụng di động: Khi Android trở thành hệ điều hành di động phổ biến nhất, các nhà phát triển đòi hỏi một ngôn ngữ giúp họ viết mã nhanh hơn, an toàn hơn để cạnh tranh với sự hiện đại của ngôn ngữ Swift trên nền tảng iOS.

1.4. Sự thay đổi qua các phiên bản

- Giai đoạn hình thành và ổn định (Phiên bản 1.0 - 1.2)
    - Kotlin 1.0 (2016): Là phiên bản ổn định đầu tiên, đánh dấu việc Kotlin sẵn sàng cho các dự án thực tế với cam kết tương thích ngược lâu dài.
    - Kotlin 1.1 (2017): Giới thiệu tính năng Coroutines (dưới dạng thử nghiệm) – một bước ngoặt trong việc xử lý các tác vụ bất đồng bộ, giúp mã nguồn tránh được tình trạng "callback hell".
- Giai đoạn bùng nổ và hiện đại hóa (Phiên bản 1.3 - 1.5)
    - Kotlin 1.3 (2018): Coroutines chính thức trở nên ổn định. Phiên bản này cũng giới thiệu Kotlin/Native, cho phép biên dịch mã nguồn trực tiếp thành mã máy mà không cần máy ảo JVM.
    - Kotlin 1.4 (2020): Tập trung mạnh vào hiệu suất của bộ soạn thảo và cải thiện hệ sinh thái đa nền tảng (Kotlin Multiplatform). Giới thiệu giao diện lập trình mới cho các thư viện tiêu chuẩn.
    - Kotlin 1.5 (2021): Giới thiệu JVM Records, Sealed Interfaces và Inline Classes, giúp việc định nghĩa cấu trúc dữ liệu trở nên chặt chẽ và tối ưu bộ nhớ hơn.
- Giai đoạn tối ưu hóa và Jetpack Compose (Phiên bản 1.6 - 1.9)

    - Kotlin 1.7 - 1.9: Đây là giai đoạn tối ưu hóa cực kỳ quan trọng cho các thư viện hiện đại như Jetpack Compose và Hilt. Ra mắt trình quản lý bộ nhớ mới cho Kotlin/Native.
    - Giới thiệu Kotlin Symbol Processing (KSP): Công cụ giúp các thư viện như Hilt và Room xử lý mã nguồn nhanh hơn gấp nhiều lần so với công nghệ KAPT cũ.
    - Bắt đầu thử nghiệm trình biên dịch K2 – hứa hẹn tốc độ biên dịch nhanh gấp đôi.
- Kỷ nguyên mới (Phiên bản 2.0+)
    - Kotlin 2.0 (Ra mắt năm 2024): Đây là cột mốc lớn nhất sau phiên bản 1.0.
    - K2 Compiler chính thức trở nên ổn định: Trình biên dịch mới này không chỉ giúp tăng tốc độ build dự án mà còn giúp các công cụ phân tích mã nguồn hiểu code sâu sắc hơn.
    - Hỗ trợ tối đa cho việc phát triển giao diện bằng Jetpack Compose, giúp quá trình render và cập nhật UI mượt mà hơn đáng kể.

1.5. Hệ sinh thái (Ecosystem)

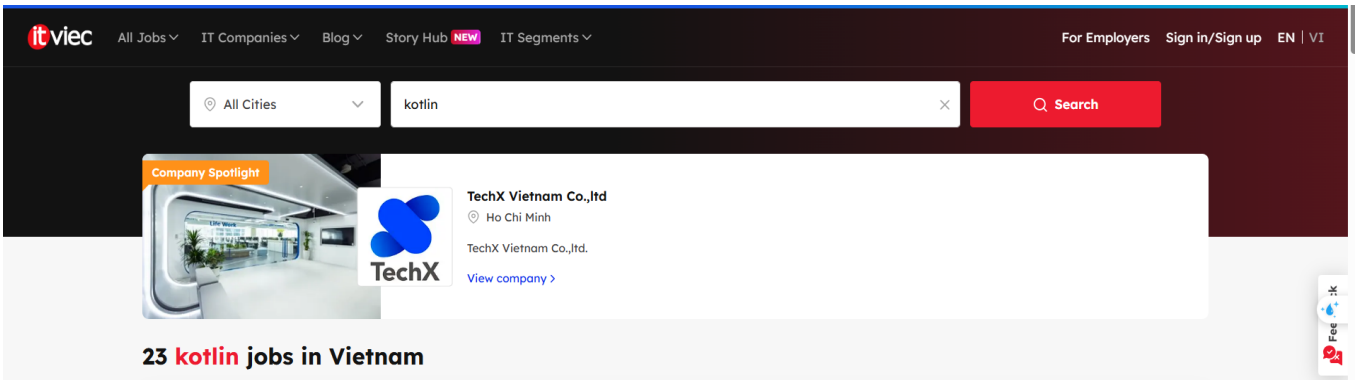
- Thư viện
    - Retrofit / Ktor Client: Thư viện tiêu chuẩn để gọi dữ liệu từ API và xử lý phản hồi từ server.
    - Coil: Thư viện tải và cache hình ảnh hiện đại, được tối ưu hóa riêng cho Jetpack Compose.
    - Room Persistence: Thư viện ORM giúp làm việc với cơ sở dữ liệu SQLite cục bộ (dùng để lưu danh sách sách yêu thích hoặc giỏ hàng).
    - Kotlin Serialization: Thư viện giúp chuyển đổi dữ liệu JSON từ API thành các đối tượng Kotlin (Data Class) một cách an toàn.

- Framework
    - Jetpack Compose: Framework hiện đại để xây dựng giao diện người dùng (UI) theo phong cách khai báo (Declarative).
    - Hilt: Framework quản lý phụ thuộc (Dependency Injection) chuẩn của Google, giúp mã nguồn sạch sẽ và dễ kiểm thử.
    - Spring Boot / Ktor: Các framework mạnh mẽ phía Backend, thường được kết hợp để xây dựng hệ thống Fullstack hoàn toàn bằng ngôn ngữ Kotlin.
- Công cụ hỗ trợ
    - Kotlin Multiplatform (KMP): Framework cho phép chia sẻ mã nguồn logic giữa Android, iOS và Web.Công cụ hỗ trợMaterial Design 3 (M3): Bộ thư viện component UI chuẩn từ Google, cung cấp các thành phần giao diện hiện đại sẵn có cho Compose.
    - Android Studio: IDE chính thống tích hợp sẵn các công cụ như Layout Inspector và Compose Preview để phát triển ứng dụng di động.
    - Gradle (Kotlin DSL): Công cụ quản lý mã nguồn, thư viện và quy trình đóng gói ứng dụng (Build tool).
    - ktlint / Detekt: Công cụ kiểm tra lỗi cú pháp và định dạng mã nguồn, giúp duy trì chất lượng code tương tự như ESLint/Prettier.
- Cộng đồng phát triển
    - Github: hàng nghìn contributor, dự án được cập nhật thường xuyên.
    - StackOverflow, Reddit, Discord: cộng đồng thảo luận, chia sẻ kinh nghiệm, giải pháp.
    - Công ty lớn áp dụng: Google, Netflix, Pinterest, Uber, Trello … góp phần khẳng định vị thế của Kotlin.

CHƯƠNG 2. CƠ HỘI NGHỀ NGHIỆP VÀ THỊ TRƯỜNG

1. Các vị trí công việc

Các vị trí công việc liên quan đến kotlin hiện nay thường tập trung chủ yếu vào lĩnh vực phát triển ứng dụng di động, do kotlin được sử dụng để xây dựng các ứng dụng di động hiệu suất cao.



1.1. Android Developer

- Đây là vị trí phổ biến và có nhu cầu tuyển dụng cao nhất. Kể từ khi Google ưu tiên Kotlin, hầu hết các dự án Android mới đều yêu cầu kỹ năng này.
- Công việc chính: Xây dựng giao diện người dùng bằng Jetpack Compose, quản lý luồng dữ liệu, tích hợp các thư viện như Hilt để quản lý mã nguồn và xử lý các tác vụ bất đồng bộ với Coroutines.
- Yêu cầu: Hiểu biết sâu về vòng đời ứng dụng Android, kiến trúc MVVM/MVI và các bộ công cụ Jetpack.

1.2. Backend Developer

- Nhờ khả năng chạy trên máy ảo Java (JVM) và tương thích hoàn toàn với các thư viện Java, Kotlin đang dần thay thế Java trong phát triển phía máy chủ.

- Công việc chính: Xây dựng các API, hệ thống Microservices và quản lý cơ sở dữ liệu bằng các framework mạnh mẽ như Spring Boot hoặc Ktor.
- Yêu cầu: Kiến thức về cơ sở dữ liệu (SQL/NoSQL), kiến trúc hệ thống và bảo mật (như Keycloak hoặc OAuth2).

1.3. Kotlin Multiplatform Developer

- Đây là vị trí đang trở thành xu hướng mới, cho phép viết mã nguồn một lần và chạy trên cả Android, iOS, Desktop và Web.
- Công việc chính: Phát triển các thư viện logic dùng chung (Business Logic) và phối hợp với các nhóm nền tảng khác để tối ưu hóa hiệu năng ứng dụng trên từng thiết bị.
- Yêu cầu: Kỹ năng tổ chức mã nguồn tốt và kinh nghiệm làm việc với Kotlin Multiplatform (KMP).

1.4. Kotlin Fullstack Developer

- Với sự hỗ trợ của Kotlin/JS và các framework backend, một lập trình viên có thể sử dụng duy nhất ngôn ngữ Kotlin để xây dựng toàn bộ một hệ thống phần mềm.
- Công việc chính: Đảm nhiệm cả việc xây dựng giao diện web (Sử dụng Kotlin/JS hoặc Compose for Web) và hệ thống xử lý dữ liệu phía sau (Backend).

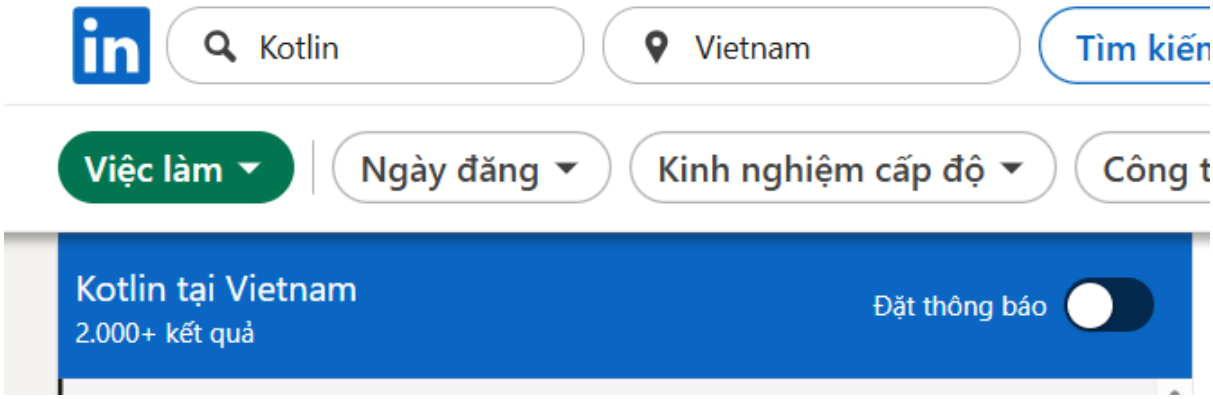
2. Nhu cầu tuyển dụng

Sự chuyển dịch mạnh mẽ từ Java sang Kotlin, đặc biệt là sau tuyên bố "Kotlin-first" của Google, đã tạo ra một làn sóng nhu cầu nhân lực rất lớn trên phạm vi toàn cầu nói chung và Việt Nam nói riêng.

2.1. Thị trường Việt Nam

Tại Việt Nam, thị trường tuyển dụng Kotlin đang trải qua giai đoạn chuyển đổi số mạnh mẽ từ các dự án kế thừa (Legacy) sang các kiến trúc hiện đại.

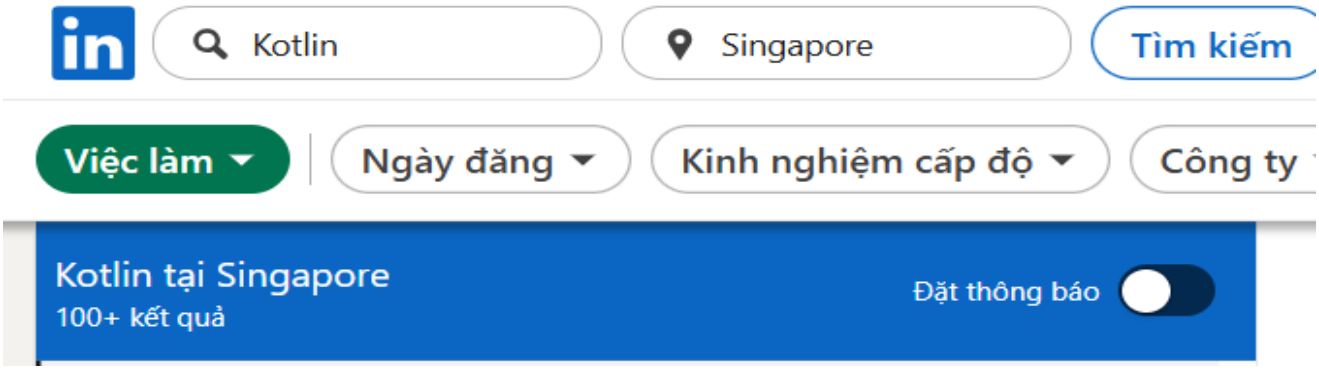
- Xu hướng tại các công ty Outsourcing: Các tập đoàn lớn như FPT Software, NashTech hay KMS Solutions liên tục tuyển dụng lập trình viên Kotlin để phục vụ các dự án từ đối tác nước ngoài. Yêu cầu hiện nay không chỉ dừng lại ở ngôn ngữ mà còn đi kèm với các công nghệ như Jetpack Compose và Hilt.
- Sự bùng nổ của các "Super App" nội địa: Các công ty sản phẩm (Product) hàng đầu tại Việt Nam như MoMo, ZaloPay, Shopee, Tiki hay các hệ thống ngân hàng số (Techcombank, VPBank) đều đã chuyển dịch sang Kotlin để xây dựng các tính năng phức tạp, yêu cầu độ ổn định và bảo mật cao.
- Phân khúc lập trình viên:
    - Intern/Fresher: Nhu cầu thực tập sinh rất lớn tại các Lab công nghệ, mức lương khởi điểm dao động từ 8.000.000 – 12.000.000 VNĐ.
    - Junior/Middle: Đây là phân khúc có nhu cầu tuyển dụng cao nhất, yêu cầu ứng viên thành thạo Coroutines và kiến trúc MVVM, mức lương từ 18.000.000 – 35.000.000 VNĐ.
    - Senior/Architect: Các vị trí đòi hỏi khả năng tối ưu hóa hệ thống và triển khai Clean Architecture, mức lương có thể vượt mốc 50.000.000 VNĐ hoặc nhận lương theo USD.



2.2. Thị trường thế giới

Trên bình diện quốc tế, Kotlin không còn là một ngôn ngữ "mới nổi" mà đã trở thành tiêu chuẩn bắt buộc trong phát triển ứng dụng di động hiện đại.

- Sự thống trị trong mảng Android: Theo thống kê từ Google, hơn 95% trong số 1.000 ứng dụng hàng đầu trên Play Store hiện được viết bằng Kotlin. Điều này khiến nhu cầu tuyển dụng lập trình viên Kotlin luôn ở mức cao và ổn định.
- Mở rộng sang hệ sinh thái đa nền tảng: Sự ra đời của Kotlin Multiplatform (KMP) đã khiến các công ty công nghệ lớn (như Netflix, McDonald's, VMware) săn đón các kỹ sư Kotlin để tối ưu hóa quy trình phát triển ứng dụng trên cả iOS và Android với một bộ mã nguồn duy nhất.
- Mức lương hấp dẫn: Theo báo cáo của Stack Overflow, Kotlin thường xuyên nằm trong nhóm các ngôn ngữ lập trình có mức lương cao nhất. Tại các thị trường như Mỹ hoặc Châu Âu, mức lương cho một kỹ sư Kotlin kinh nghiệm thường dao động từ $100.000 đến $160.000/năm.



3. Mức lương tham khảo

3.1. Intern (0 - Dưới 6 tháng)

Mức lương: 3.000.000 – 6.000.000 VNĐ/tháng

Kinh nghiệm: 0 - Dưới 6 tháng

Kỹ năng cứng (Hard Skills):

- Biết cú pháp Kotlin cơ bản (val/var, null safety).

- Biết sử dụng cơ bản Jetpack Compose để vẽ UI.
- Hiểu khái niệm cơ bản về Hilt/DI.
- Biết sử dụng Git cơ bản.

Kỹ năng mềm (Soft Skills):

- Tinh thần học hỏi: Chủ động hỏi, ghi chép và tự tổng hợp kiến thức.
- Tuân thủ: Chấp nhận và làm theo hướng dẫn chi tiết, chính xác.
- Ngoại ngữ: Đủ để đọc hiểu tài liệu kỹ thuật cơ bản.

Thực tế cho thấy, số lượng công việc dành cho ứng viên chưa có nhiều kinh nghiệm hiện còn khá hạn chế, mức lương khởi điểm dao động khoảng 3 – 6 triệu VNĐ/tháng tùy công ty và yêu cầu công việc.

3.2. Fresher (Dưới 1 năm)

Mức lương: 6.000.000 – 12.000.000 VNĐ/tháng

Kinh nghiệm: Dưới 1 năm (sau giai đoạn Intern)

Kỹ năng cứng (Hard Skills):

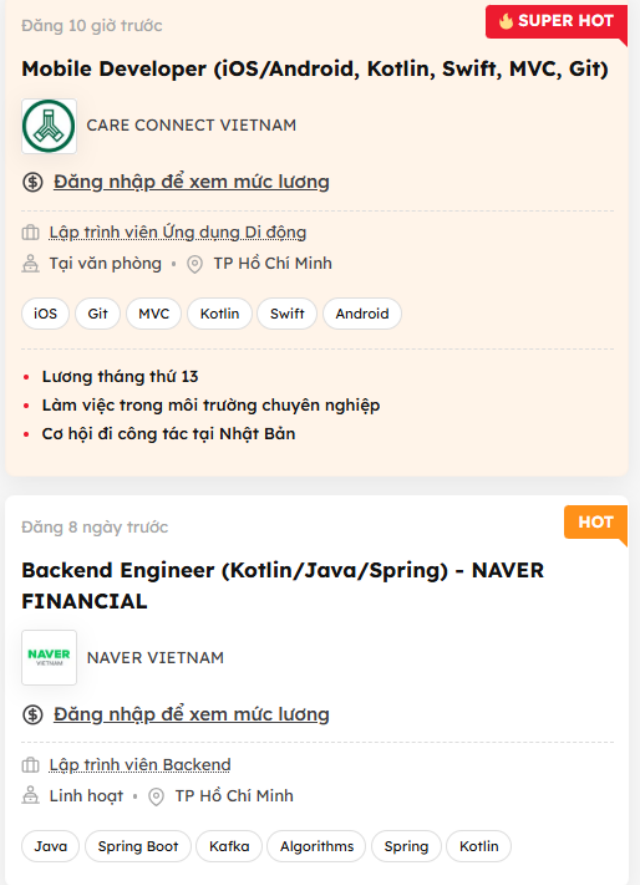
- Kotlin & xử lý bất đồng bộ:
  - Thành thạo Kotlin cơ bản: data class, sealed class, extension function.
  - Có kinh nghiệm sử dụng Coroutine (launch, async, suspend).
  - Hiểu Flow cơ bản là lợi thế.
  - Hiểu các scope như viewModelScope, lifecycleScope.
- Kiến trúc & Clean Code:
  - Có kinh nghiệm làm việc với MVVM hoặc MVC.
  - Biết tách layer: UI – ViewModel – Repository – Data Source.
  - Biết xử lý UI State (Loading / Success / Error).
  - Code rõ ràng, dễ maintain và có khả năng refactor.
- Networking & Data:
  - Sử dụng Retrofit / OkHttp để gọi API.

- Hiểu RESTful API.
- Biết xử lý timeout, error và parsing JSON.
- Có kinh nghiệm với Room hoặc SQLite là lợi thế.

Kỹ năng mềm (Soft Skills):

- Chịu trách nhiệm: Hoàn thành các task nhỏ được giao và đảm bảo code chạy.
- Tìm kiếm/Debug: Tự debug các lỗi phổ biến trước khi nhờ hỗ trợ.

1. Tư duy code sạch: Bắt đầu có ý thức về việc viết code dễ đọc.



3.3. Junior Developer (1 - 2 năm)

Mức lương: 15.000.000 – 25.000.000 VNĐ/tháng

Kinh nghiệm: 1 - 2 năm

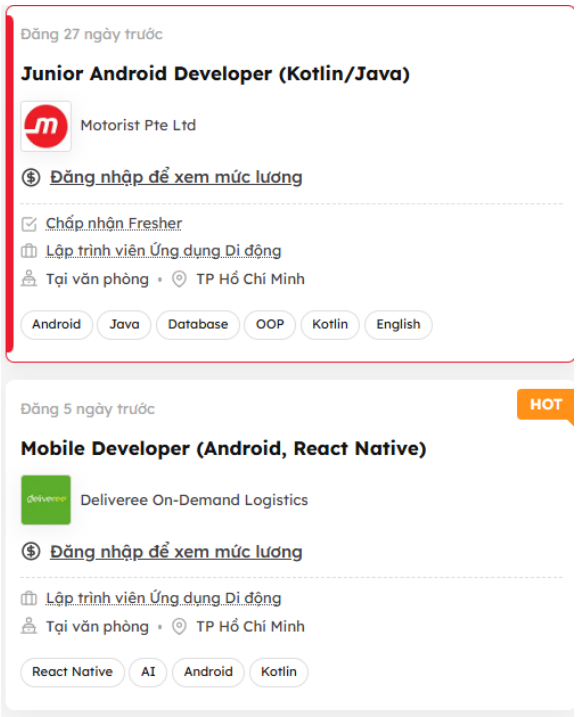
Kỹ năng cứng (Hard Skills):

- Hiểu biết về các nguyên tắc cơ bản của Java/Kotlin, Android Studio và Android SDK.

- Làm quen với các nguyên tắc SOLID, OOP, cấu trúc dữ liệu và kiến trúc sạch (MVVM).
- Kinh nghiệm làm việc với cơ sở dữ liệu như SQLite, Room và Realm.
- Hiểu biết về tích hợp API RESTful và xử lý dữ liệu JSON.
- Hiểu biết thành thạo về các công cụ kiểm soát phiên bản mã, chẳng hạn như Git.
- Chú ý đến chi tiết và có tư duy phân tích giải quyết vấn đề.
- Tiếp xúc với Jetpack Compose, Retrofit2 / OkHttp3, RxJava hoặc Google Maps SDK.
- Hiểu biết cơ bản về Nhắn tin Firebase, Crashlytics hoặc Cấu hình từ xa.
- Làm quen với thiết kế Material 3, Coroutines Flow hoặc thử nghiệm JUnit/Espresso.

Kỹ năng mềm (Soft Skills):

- Độc lập và chủ động: Tự giải quyết hầu hết các vấn đề ở mức trung bình và cần ít sự giám sát từ Senior.
- Ước tính: Có khả năng ước tính thời gian làm task (effort estimation) hợp lý.
- Ngoại ngữ: Đọc hiểu tài liệu chuyên sâu (ví dụ: RFC, tài liệu kỹ thuật nâng cao).



3.4. Mid-Level Developer (2 - 4 năm)

Mức lương: 24.000.000 – 40.000.000 VNĐ/tháng

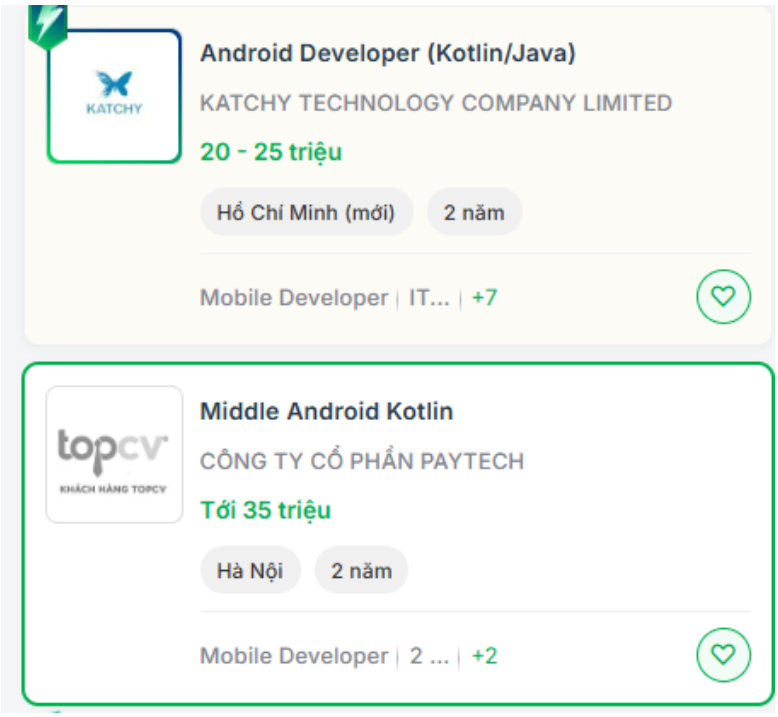
Kinh nghiệm: 2 – 4 năm

Kỹ năng cứng (Hard Skills):

- Thành thạo Kotlin (Java là một lợi thế).
- Kinh nghiệm với Jetpack components (LiveData, ViewModel, Navigation, Room, DataStore, v.v.).
- Hiểu biết vững chắc về các mẫu kiến trúc như MVVM, MVI hoặc Clean Architecture.
- Thành thạo với dependency injection (ví dụ: Hilt, Dagger), networking (Retrofit/OkHttp) và reactive programming (ví dụ: Coroutines, Flow, RxJava).
- Có kinh nghiệm làm việc với quy trình Git, CI/CD và các công cụ DevOps hiện đại.
- Hiểu biết về các biện pháp bảo mật di động, đặc biệt có liên quan trong các ứng dụng ngân hàng (ví dụ: mã hóa dữ liệu, lưu trữ an toàn, xác thực sinh trắc học).

Kỹ năng mềm (Soft Skills):

- Phân tích & giải pháp: Phân tích được nguyên nhân gốc rễ (Root Cause) của vấn đề và đề xuất giải pháp bền vững cho hệ thống.
- Mentoring: Hướng dẫn và hỗ trợ các Junior trong team về kỹ thuật và quy trình.
- Ngoại ngữ: Tiếng Anh giao tiếp cơ bản (Tham gia họp, thảo luận task, phản hồi code review qua văn bản) là lợi thế lớn để đạt mức lương cao hơn 30M.



Đây là giai đoạn trung gian trong lộ trình nghề nghiệp: ứng viên đã vượt qua Junior, đủ khả năng đảm nhận những phần việc độc lập hoặc cả một module, nhưng chưa đạt đến vai trò định hướng như Senior. Trên thực tế, số lượng JD ghi rõ "Middle" thường ít, vì nhiều doanh nghiệp gộp nhóm này vào Senior hoặc tuyển theo khung kinh nghiệm.

3.5. Senior Developer (4 - 6 năm)

Mức lương: 40.000.000 – 65.000.000+ VNĐ/tháng

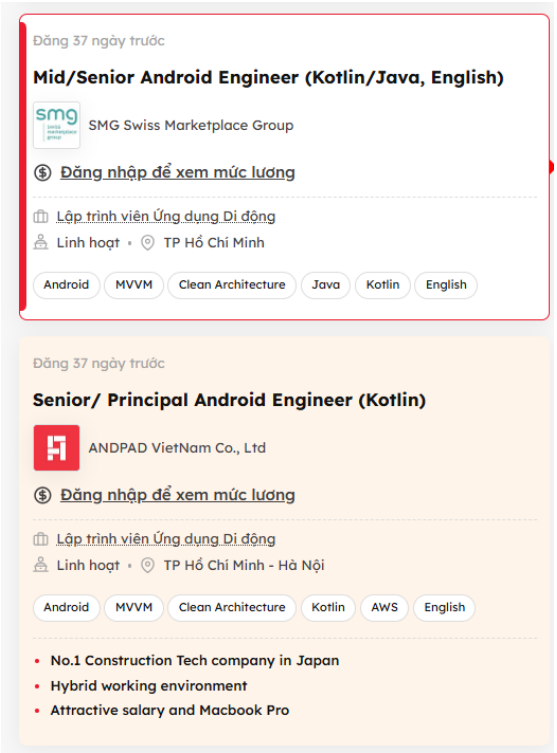
Kinh nghiệm: 4 - 6 năm

Kỹ năng cứng (Hard Skills):

- Chuyên gia kiến trúc: Thiết kế kiến trúc cho các ứng dụng quy mô lớn.
- Bảo mật & Scale: Chuyên gia tối ưu hóa hiệu năng chuyên sâu và có kinh nghiệm xử lý các vấn đề bảo mật trong môi trường Production.
- Full-stack/System: Thành thạo một Backend Stack (Kotlin/java,spring…) và có khả năng thiết kế Database Schema (SQL/NoSQL).
- DevOps: Thiết lập và duy trì quy trình CI/CD, Monitoring, Logging .

Kỹ năng mềm (Soft Skills):

- Lãnh đạo kỹ thuật: Dẫn dắt Code Review, đưa ra các tiêu chuẩn kỹ thuật cho toàn team.
- Tư duy chiến lược: Hiểu rõ mục tiêu kinh doanh và đề xuất giải pháp kỹ thuật tối ưu chi phí và thời gian.
- Ngoại ngữ (Yếu tố quyết định): Tiếng Anh lưu loát (thảo luận kỹ thuật sâu, bảo vệ quan điểm kiến trúc, làm việc trực tiếp với khách hàng/đối tác nước ngoài).



So với Middle, Senior không chỉ đảm nhận phần việc độc lập mà còn có trách nhiệm dẫn dắt đội nhóm nhỏ, review code và đưa ra giải pháp kỹ thuật cho dự án. Trên thị trường Việt Nam, JD Senior xuất hiện phổ biến hơn Middle vì nhiều doanh nghiệp trực tiếp tuyển ứng viên đã có kinh nghiệm cao để giảm chi phí đào tạo.

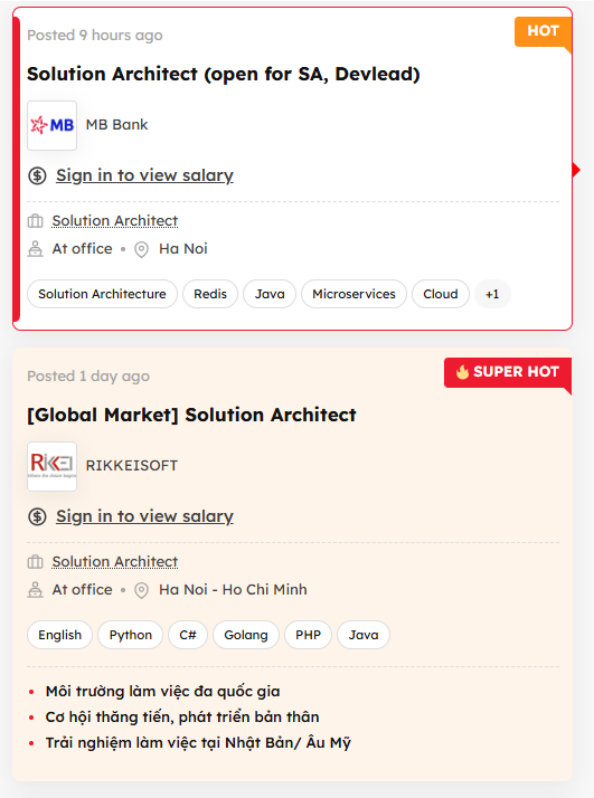
3.6. Tech Lead/Architect (6+ năm)

Mức lương: 65.000.000 – 100.000.000+ VNĐ/tháng
Kinh nghiệm: 6 năm trở lên
Kỹ năng cứng (Hard Skills):

- Định hướng công nghệ: Lựa chọn Stack công nghệ tổng thể, chịu trách nhiệm về Roadmap kỹ thuật của sản phẩm.
- Quản lý vận hành: Chịu trách nhiệm về SLAs (Service Level Agreements), đảm bảo hệ thống hoạt động ổn định và có khả năng phục hồi sau thảm họa (Disaster Recovery).
- Tích hợp đa hệ thống: Thiết kế các giải pháp tích hợp với các hệ thống Legacy/Microservices phức tạp.

Kỹ năng mềm (Soft Skills):

- Quản lý đội ngũ: Quản lý tiến độ dự án (Scrum/Agile), phân công task, đánh giá hiệu suất của lập trình viên.
- Giao tiếp chiến lược: Thương lượng và trình bày giải pháp kỹ thuật cho quản lý cấp cao hoặc ban lãnh đạo.
- Ngoại ngữ: Sử dụng thành thạo trong mọi tình huống công việc (phỏng vấn, đào tạo, viết proposal).



Ở cấp độ này, JD thường được mô tả cụ thể và rõ ràng, nhấn mạnh cả năng lực kỹ thuật (thiết kế kiến trúc, tối ưu hiệu năng, bảo đảm chất lượng sản phẩm) và kỹ năng quản lý đội nhóm. Doanh nghiệp biết chính xác điều họ cần, đồng thời tin tưởng rằng ứng viên ở cấp này sẽ hiểu ngay yêu cầu công việc. Theo quan sát, số lượng tin tuyển Tech Lead có phần nhiều hơn so với Middle, phản ánh nhu cầu cao về nhân sự chủ chốt giúp định hướng dự án.

4. Các công ty/sản phẩm tiêu biểu đang sử dụng

Kotlin không chỉ là một ngôn ngữ lập trình được ưa chuộng trong cộng đồng phát triển di động, mà còn đã chứng minh tính ứng dụng thực tiễn thông qua việc được nhiều doanh nghiệp công nghệ hàng đầu thế giới và trong nước áp dụng vào các sản phẩm then chốt. Điều này khẳng định vị thế của Kotlin trong việc tối ưu hóa hiệu suất phát triển, tăng tính an toàn cho mã nguồn và mang lại trải nghiệm ứng dụng mượt mà, ổn định.

4.1. Quốc tế

- Google (Android & YouTube): Là đơn vị thúc đẩy mạnh mẽ nhất, Google đã chuyển dịch toàn bộ hệ sinh thái Android sang ưu tiên Kotlin (Kotlin-first). Các ứng dụng như

YouTube, Google Drive và bản thân hệ điều hành Android sử dụng Kotlin để giảm thiểu 33% lỗi crash hệ thống nhờ tính năng an toàn với giá trị null (Null Safety).

- Netflix: Ứng dụng xem phim trực tuyến hàng đầu thế giới sử dụng Kotlin cho ứng dụng Android của mình để tối ưu hóa việc truyền tải dữ liệu (streaming) và xây dựng các dịch vụ phía Backend, giúp hệ thống chịu tải lớn nhưng vẫn duy trì được sự ổn định tuyệt đối.
- Uber: Với mạng lưới người dùng và tài xế khổng lồ, Uber ứng dụng Kotlin để xây dựng các ứng dụng di động có quy mô lớn. Việc sử dụng Kotlin giúp đội ngũ kỹ sư của Uber viết mã nhanh hơn, dễ bảo trì và đảm bảo tính an toàn cao cho các giao dịch và định vị thời gian thực.
- Pinterest: Là một trong những doanh nghiệp lớn đầu tiên chuyển đổi từ Java sang Kotlin. Pinterest sử dụng ngôn ngữ này để cải thiện hiệu suất ứng dụng và giảm đáng kể lượng mã nguồn thừa (boilerplate code), giúp quy trình cập nhật tính năng mới trở nên linh hoạt hơn.

4.2. Việt Nam

- MoMo: Siêu ứng dụng thanh toán hàng đầu Việt Nam sử dụng Kotlin để phát triển hệ thống Android, giúp đảm bảo tính bảo mật cực cao cho các giao dịch tài chính và mang lại giao diện mượt mà cho hàng triệu người dùng hàng ngày.
- Zalo: Ứng dụng nhắn tin phổ biến nhất Việt Nam đã ứng dụng Kotlin để tối ưu hóa tốc độ gửi nhận tin nhắn và xử lý các tác vụ bất đồng bộ phức tạp, giúp ứng dụng hoạt động ổn định trên nhiều dòng thiết bị khác nhau.
- Tiki/Shopee: Các sàn thương mại điện tử lớn tại Việt Nam áp dụng Kotlin kết hợp với các bộ thư viện hiện đại để xây dựng ứng dụng mua sắm, giúp tối ưu tốc độ tải hình ảnh sản phẩm và cải thiện tỷ lệ chuyển đổi thông qua trải nghiệm người dùng mượt mà.

Shopee	TIKI
Zalo pay	topcv

NỘI DUNG CỐT LÕI VÀ THỰC HÀNH

CHƯƠNG 2: CÀI ĐẶT MÔI TRƯỜNG VÀ "HELLO WORLD"

Lý thuyết

Môi trường phát triển

Để bắt đầu với Kotlin, trước hết cần chuẩn bị môi trường phát triển phù hợp, Kotlin là ngôn ngữ chạy trên máy ảo Java (JVM), do đó yêu cầu bắt buộc là phải cài đặt Java Development Kit (JDK). Ngoài ra, chúng ta cần một công cụ quản lý thư viện và tự động hoá build như Gradle hoặc Maven; trong đó, Gradle với Kotlin DSL (Domain Specific Language) là lựa chọn phổ biến khi làm việc với Kotlin.

Java Development Kit (JDK)

JDK cung cấp môi trường chạy (JRE) và các công cụ phát triển (trình biên dịch java, javac, ...) cần thiết để biên dịch và chạy các chương trình trên JVM.

- **Chức năng chính:** Biên dịch mã nguồn Kotlin thành bytecode Java thực thi trên JVM.
- **Phiên bản khuyến khích:** nên sử dụng các phiên bản JDK LTS (Long Term Support) như JDK 11, JDK 17 hoặc JDK 21 để đảm bảo tính ổn định và hiệu quả.
- **Kiểm tra cài đặt: sau khi cài đặt**, mở terminal và gõ java –version và javac –version để xác nhận.

Gradle

Gradle là một công cụ tự động hoá build, được sử dụng rộng rãi trong hệ sinh thái Kotlin. Nó giúp quản lý thư viện (dependencies), biên dịch, chạy kiểm thử và đóng gói ứng dụng.

- **Có thể cài đặt riêng** hoặc thông qua sử dụng **Wrapper (gradlew)** đi kèm dự án, giúp đồng bộ phiên bản giữa các máy.
- **Kotlin LTS** cho phép viết các tệp cấu hình build bằng Kotlin thay vì Groovy, tận dụng được các tính năng của ngôn ngữ (type-safety, code completion).

Hệ điều hành

Kotlin hỗ trợ đa nền tảng, bao gồm Windows, macOS và Linux. Tuỳ theo hệ điều hành, cách cài đặt JDK có thể khác nhau:

- **Trên Windows,** tải toàn bộ từ trang chủ của nhà cung cấp (Oracle, Adoptium, ...).
- **Trên macOs/Linux,** có thể sử dụng trình quản lý gói như Homebrew (macOS), SDKMAN (Linux/macOS) để dễ dàng cài đặt và chuyển đổi giữa các phiên bản SDK.

Công cụ soạn thảo

IntelliJ IDEA

IntelliJ IDEA, phát triển bởi JetBrains (cũng là cha đẻ của Kotlin), là môi trường phát triển tích hợp (IDE) mạnh mẽ nhất dành cho Kotlin. Phiên bản Community Edition hoàn toàn miễn phí và cung cấp đầy đủ tính năng cần thiết.

Ưu điểm nổi bật:

- **Hỗ trợ Kotlin nguyên bản:** code completion, gợi ý kiểu dữ liệu, refactoring thông minh.
- **Tích hợp build tool:** nhập trực tiếp dự án Gradle hoặc Maven, tự động tải dependencies.
- **Debugger mạnh mẽ:** cho phép đặt breakpoint, theo dõi giá trị biến, thực thi từng dòng.
- **Giao diện thân thiện:** dễ dàng tạo mới project Kotlin với nhiều mẫu có sẵn (console, web, ...).

Ngoài IntelliJ IDEA, một số IDE khác như Visual Studio Code hoặc Android Studio cũng có thể sử dụng, tuy nhiên đôi khi chúng không hổ trợ cộng đồng mạnh mẽ như IntelliJ IDEA.

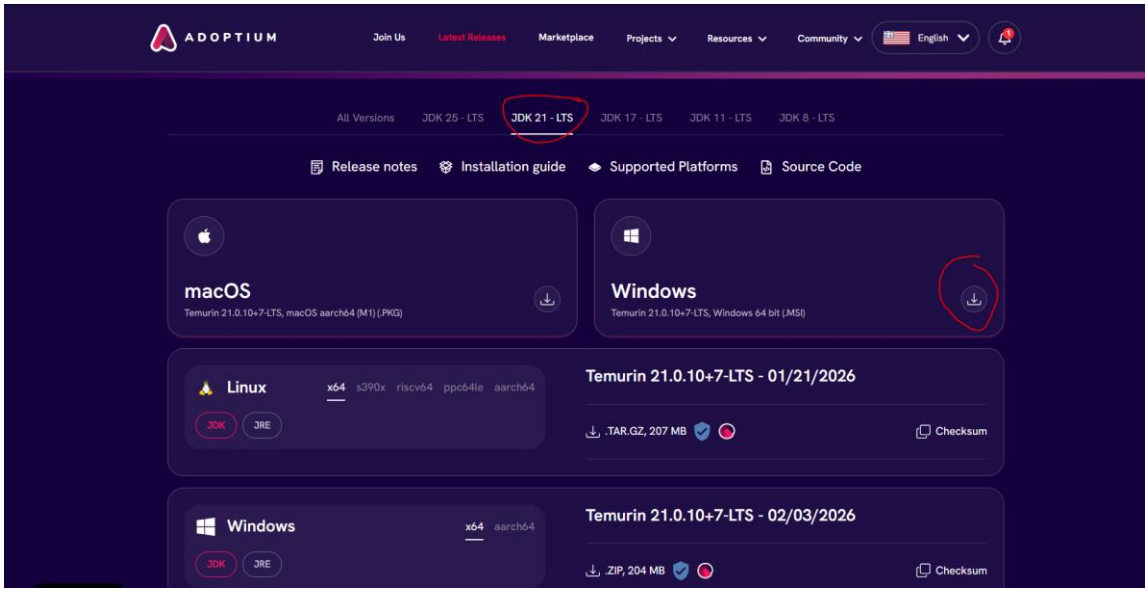
Hướng dẫn và thực hành

Để bắt đầu với Kotlin, cần tiến hành một số bước cài đặt cơ bản nhằm thiết lập môi trường làm việc và khởi tạo dự án đầu tiên. Toàn bộ quy trình có thể thực hiện trực tiếp trên máy tính cá nhân, không yêu cầu cấu hình phức tạp.

Bước 1: cài đặt JDK

Truy cập vào trang Apdotium.net (hoặc trang chủ Oracle) vào tải xuống phiên bản JDK LTS mới nhất (ví dụ JDK 17). Chạy tệp và làm theo hướng dẫn. Sau khi chạy xong, mở terminal và kiểm tra phiên bản bằng lệnh:

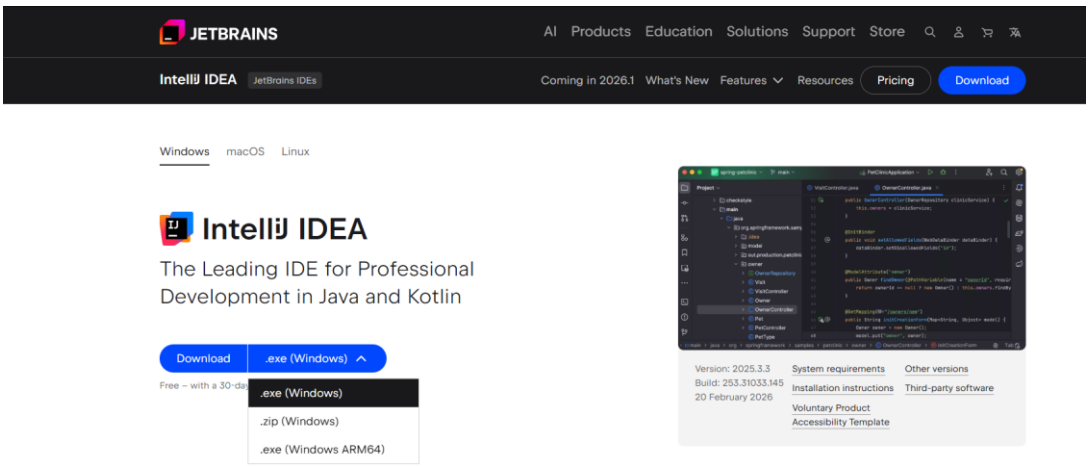
```
java -version
javac -version
```

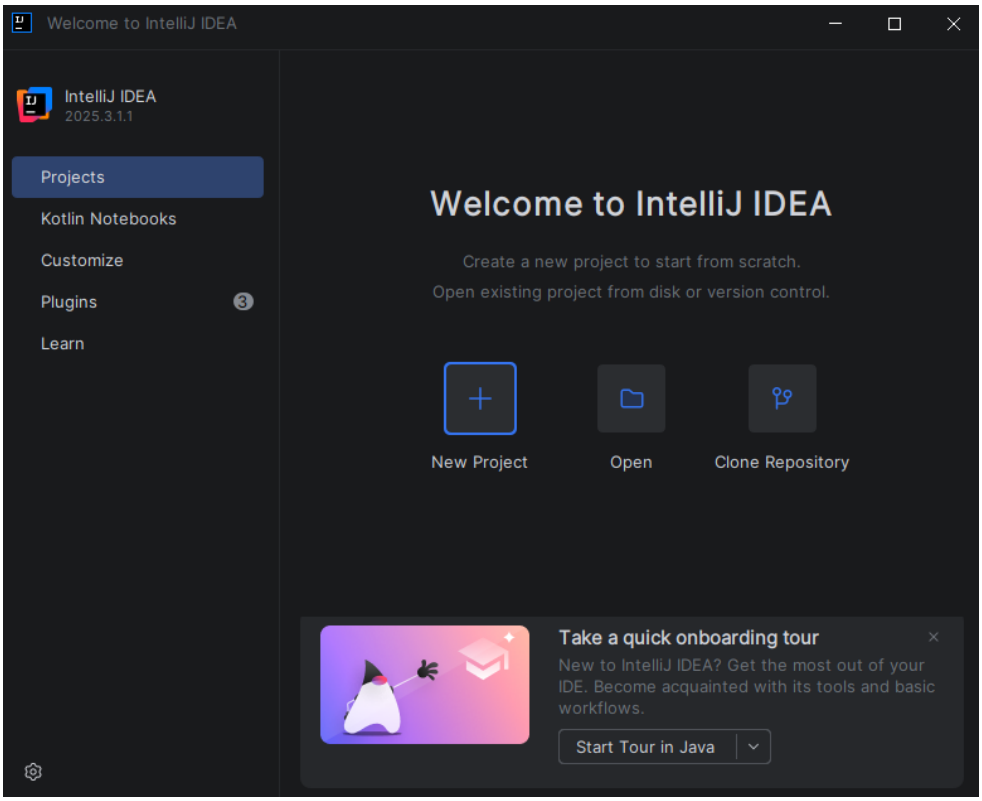


```
$ java -version
openjdk version "17.0.10" 2024-01-16 LTS
OpenJDK Runtime Environment Temurin-17.0.10+7 (build 17.0.10+7)
OpenJDK 64-Bit Server VM Temurin-17.0.10+7 (build 17.0.10+7, mixed mode)
```

Bước 2: Cài đặt IntelliJ IDEA

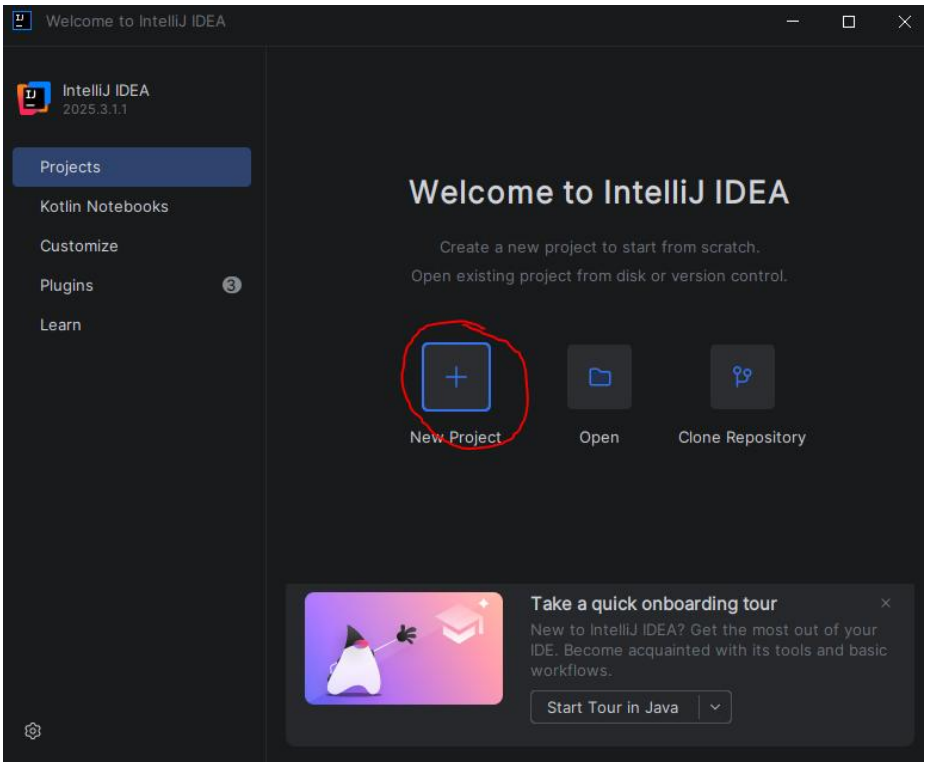
Truy cập trang web jetbrains.com/idea/download và tải xuống phiên bản Community Edition phù hợp với hệ điều hành. Chạy tệp cài đặt, chọn các tuỳ chọn mặc định)có thể liên kết đến menu Start/Desktop). Sau khi cài đặt, khởi động IntelliJ IDEA.



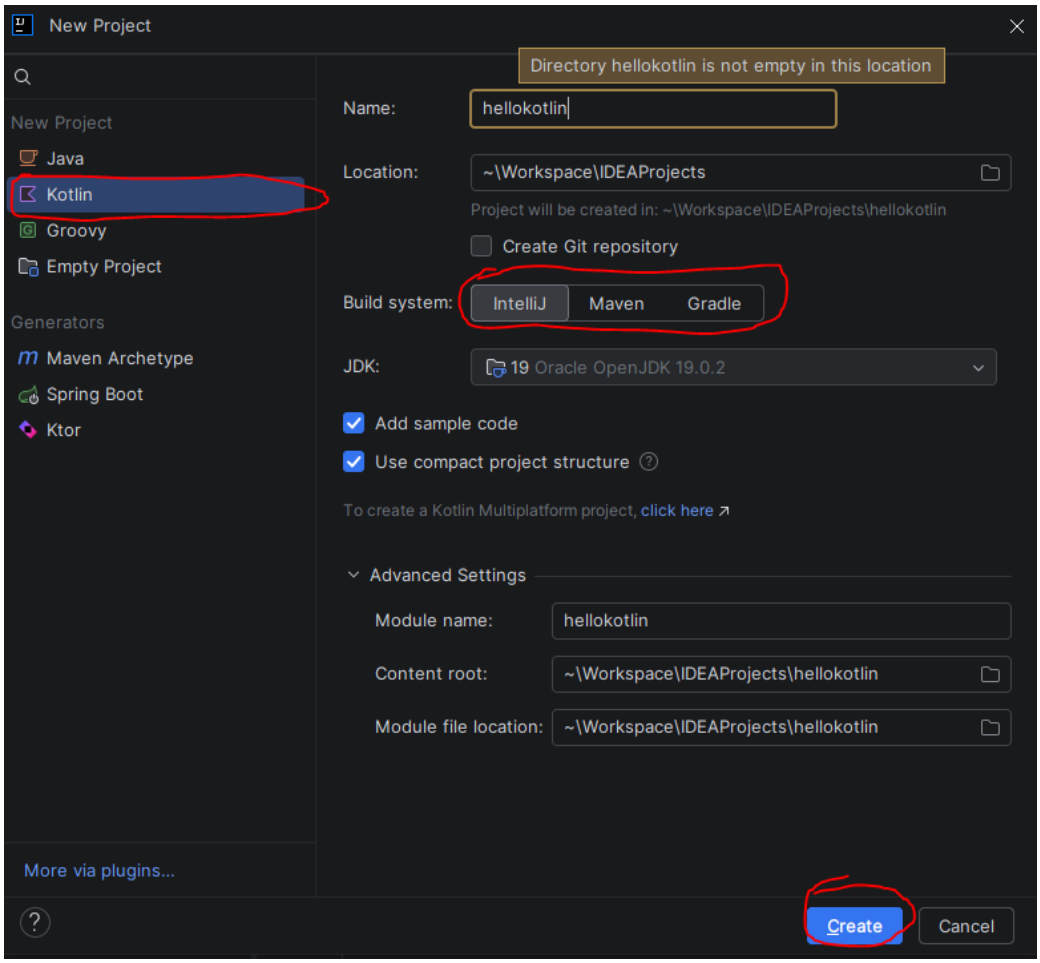


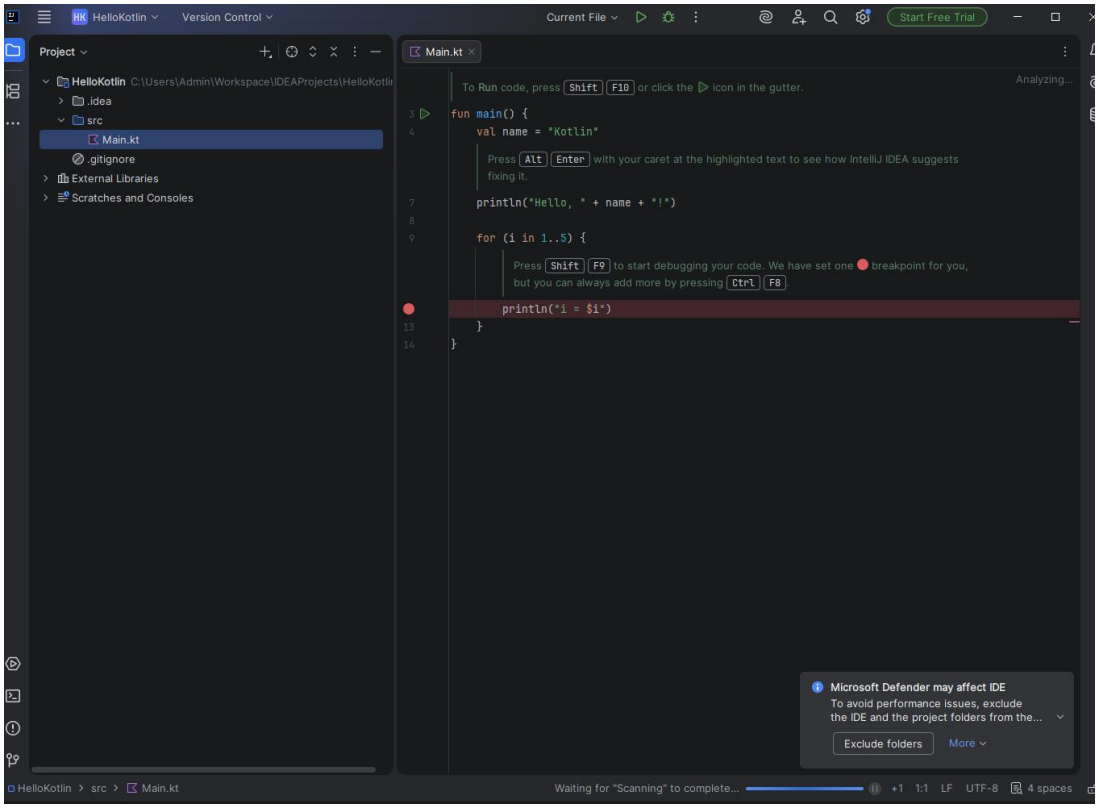
Bước 3: Tạo dự án Kotlin mới

Sau khi đã cài đặt xong IntelliJ IDEA mà mở ứng dụng lần đầu, ta sẽ xuất hiện ở màn hình chào mừng. Trên màn hình chào mừng của IntelliJ IDEA, chọn nút New Project.



Trong cửa sổ hiện ra, chọn Kotlin ở cột bên trái, sau đó chọn Build System. Đặt tên dự án (Ví dụ trong hình: HelloKotlin). Trong mục Project SDK, chọn JDK đã cài (Nếu chưa thấy, nhấn Add JDK và dẫn đến thư mục cài đặt). Nhấn Create, IntelliJ IDEA sẽ tạo cấu trúc dự án với một tệp Main.kt mặc định trong thư mục src/main/kotlin.


New Project
Directory hellokotlin is not empty in this location
New Project
Java
Kotlin
Groovy
Empty Project
Generators
Maven Archetype
Spring Boot
Ktor
More via plugins...
Name:
hellokotlin
Location:
~\Workspace\IDEAProjects
Project will be created in: ~\Workspace\IDEAProjects\hellokotlin
Create Git repository
Build system:
IntelliJ
Maven
Gradle
JDK:
19 Oracle OpenJDK 19.0.2
Add sample code
Use compact project structure
To create a Kotlin Multiplatform project, click here
Advanced Settings
Module name:
hellokotlin
Content root:
~\Workspace\IDEAProjects\hellokotlin
Module file location:
~\Workspace\IDEAProjects\hellokotlin
Create
Cancel

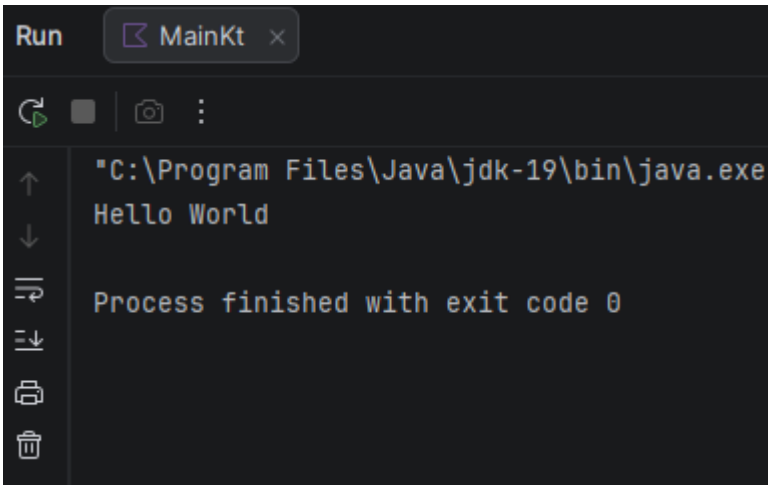

HelloKotlin
Version Control
Current File
Start Free Trial
Project
HelloKotlin C:\Users\Admin\Workspace\IDEAProjects\HelloKotlin
.idea
src
Main.kt
.gitignore
External Libraries
Scratches and Consoles
Main.kt
To Run code, press Shift F10 or click the icon in the gutter.
fun main() {
val name = "Kotlin"
Press Alt Enter with your caret at the highlighted text to see how IntelliJ IDEA suggests fixing it.
println("Hello, " + name + "!")
for (i in 1..5) {
Press Shift F9 to start debugging your code. We have set one breakpoint for you, but you can always add more by pressing Ctrl F8.
println("i = $i")
}
}
Microsoft Defender may affect IDE
To avoid performance issues, exclude the IDE and the project folders from the...
Exclude folders
More
HelloKotlin > src > Main.kt
Waiting for "Scanning" to complete...
1:1
LF
UTF-8
4 spaces

Bước 4: Viết mã nguồn “Hello, Wold!” và chạy ứng dụng

Mỡ tệp Main.kt (hoặc tệp .kt vừa được tạo). Xoá nội dung cũ nếu có và viết đoạn mã sau:

```
fun main(){
    println(""Hello, World!")
}
```

Nhấp chuột phải vào bất kì đâu trong tệp Main.kt, chọn Run ‘MainKt’ (Hoặc nhấn tổ hợp phím Ctrl + Shift + F10 trên Windows/Linux, ^ + Shift + R trên macOS). Quan sát cửa sổ run ở cuối màn hình. Kết quả hiển thị sẽ là:



CHƯƠNG 3: NGÔN NGỮ LẬP TRÌNH KOTLIN

Lý thuyết

Biến và Kiểu dữ liệu

Trong Kotlin, biến được khai báo theo hai cách dứt khoát thông qua từ khóa val và var, phản ánh tư tưởng thiết kế hướng đến sự an toàn và rõ ràng trong mã nguồn.

Khai báo biến

- val (value – giá trị): Khai báo hằng số bất biến (immutable). Sau khi khởi tạo, giá trị không thể thay đổi. Tương đương với final trong Java.
- var (variable – biến): Khai báo biến có thể thay đổi giá trị (mutable) trong suốt vòng đời của nó

Ví dụ khai báo biến

```
Val bookTitle: String = "Kotlin Programming"  // Không thể gán lại
Var bookPricee: Double = 150.000 // Có thể thay đổi
bookPrice = 120.000 // Hợp lệ
```

> *Lưu ý: Kotlin có khả năng suy luận kiểu (Type Inference). Lập trình viên không cần khai báo tường minh kiểu dữ liệu nếu giá trị đã đủ rõ ràng. Ví dụ: val name = "Kotlin" – trình biên dịch tự suy ra kiểu String.*

Các kiểu dữ liệu nguyên thủy trong Kotlin

Không giống Java, Kotlin không phân biệt kiểu nguyên thủy (primitive) và kiểu đối tượng (object) ở cấp ngôn ngữ. Toàn bộ đều là đối tượng nhưng được trình biên dịch tối ưu hóa thành kiểu nguyên thủy của JVM ở cấp bytecode.

- **Số nguyên:** Byte, Short, Int, Long
- **Số thực:** Float, Double
- **Ký tự:** Char
- **Logic:** Boolean (true / false)
- **Chuỗi ký tự:** String – hỗ trợ String Templates với cú pháp ${…}

Ví dụ minh họa String Templates:

```
val bookName = "Lập trình Kotlin"
val price = 200.000
println("Sách: $bookName - Giá: ${price}đ")
// Kết quả: Sách: Lập trình Kotlin - Giá: 200000.0đ
```

Hàm (Functions)

Hàm trong Kotlin được khai báo bằng từ khóa fun. Kotlin hỗ trợ nhiều dạng hàm linh hoạt, từ hàm thông thường đến hàm biểu thức đơn dòng, giúp rút ngắn và tăng tính biểu đạt của mã nguồn.

Cú pháp cơ bản

```
fun tên_hàm(tham_số: KiểuDữLiệu): KiểuTrảVề {
    // Thân hàm
    return giá_trị
}
```

Ví dụ hàm tính giá sau giảm:

```
fun tinhGiaSauGiam(gia: Double, phanTramGiam: Int): Double {
    return gia * (1 - phanTramGiam / 100.0)
}
```

Hàm biểu thức đơn (Single-expression Function)

Khi than hàm chỉ gồm một biểu thức, có thể viết gọn bằng toán tử gán (=):

```
fun tinhGiaSauGiam(gia: Double, phanTramGiam: Int): Double =
    gia * (1 - phanTramGiam / 100.0)
```

Tham số mặc định và tham số có tên (Default & Named Parameters)

Kotlin cho phép đặt giá trị mặc định cho tham số, giúp loại bỏ việc overload hàm không cần thiết như trong Java:

```
fun taoLoi(thongBao: String, maLoi: Int = 400): String {
    return "[Lỗi $maLoi] $thongBao"
}
// Gọi hàm không cần truyền maLoi
println(taoLoi("Không tìm thấy sách")) // [Lỗi 400] Không tìm
   thấy sách
```

Lớp và Đối tượng (Classes & Objects)

Kotlin là ngôn ngữ lập trình hướng đối tượng hoàn toàn. Cú pháp khai báo lớp ngắn gọn và mạnh mẽ hơn Java đáng kể.

Khai báo lớp cơ bản

Constructor chính (Primary Constructor) được đặt ngay sau tên lớp, giúp loại bỏ hoàn toàn các phương thức Getter và Setter rườm rà của Java:

```
class Book(val title: String, val author: String, var price:
   Double) {
```

```
    fun displayInfo(): String {
        return "$title - $author (${price}đ)"
    }
}
val book = Book("Kotlin Coroutines", "Roman Elizarov",
   350000.0)
println(book.displayInfo())
```

Kế thừa (Inheritance)

Trong Kotlin, mọi lớp đều mặc định là final (không thể kế thừa). Để cho phép kế thừa, phải đánh dấu lớp bằng từ khóa open:

```
open class Vehicle(val brand: String, val speed: Int)

class ElectricVehicle(brand: String, speed: Int, val
   batteryCapacity: Int)
    : Vehicle(brand, speed)
```

Data Class

Data Class là một loại lớp đặc biệt trong Kotlin, được thiết kế chuyên dụng để lưu trữ dữ liệu. Khi khai báo với từ khóa data class, trình biên dịch tự động sinh ra các phương thức: equals(), hashCode(), toString(), copy().

```
data class Book(val id: Int, val title: String, val price:
   Double)

val book1 = Book(1, "Kotlin in Action", 299000.0)
val book2 = book1.copy(price = 250000.0) // Tạo bản sao với giá
   khác
println(book1) // Book(id=1, title=Kotlin in Action,
   price=299000.0)
```

> *Lưu ý: Data Class là nền tảng để xây dựng Model trong kiến trúc MVVM. Mỗi đối tượng dữ liệu như Book, Ordder, User trong đồ án đều được định nghĩa là Data Class*

Kiểm soát luồng (Control Flow)

Kotlin hiện đại hóa các câu lệnh kiểm soát luồng truyền thống. Đáng chú ý là if và when trong Kotlin không chỉ là câu lệnh (statement) mà còn là biểu thức (expression) có thể trả về giá trị.

Biểu thức when – thay thế cho switch

when trong Kotlin linh hoạt và mạnh mẽ hơn switch trong Java: có thể kiểm tra giá trị, kiểu dữ liệu, và khoảng giá trị:

```
fun xepLoaiSach(theLoai: String): String = when (theLoai) {
    "Kỹ thuật", "Lập trình" -> "Sách chuyên ngành"
    "Văn học"               -> "Sách văn học"
    else                    -> "Thể loại khác"
}
```

Vòng lập for – duyệt tập hợp

Kotlin đơn giản hóa việc duyệt danh sách và khoảng giá trị:

```
val books = listOf("Kotlin", "Android", "Compose")
for (book in books) {
    println(book)
}
// Duyệt theo chỉ số
for (index in books.indices) {
    println("$index: ${books[index]}")
}
```

Ví dụ và giải thích

Ví dụ minh họa: Hệ thống quản lý danh mục sách

Mục tiêu của ví dụ

Ví dụ "Hệ thống quản lý danh mục sách" được xây dựng nhằm áp dụng tổng hợp các khái niệm ngôn ngữ cơ bản của Kotlin đã được trình bày trong phần lý thuyết. Mục tiêu chính là minh họa cách thức hoạt động thực tế của:

- Data Class trong việc mô hình hóa dữ liệu sách.
- Hàm (function) để xử lý nghiệp vụ như lọc sách, tính tổng giá trị.
- Biểu thức when và for để kiểm soát luồng xử lý.
- Collections (List, Map) để quản lý tập hợp dữ liệu

Cấu trúc ví dụ

Ví dụ được tổ chức trong một file Kotlin duy nhất, bao gồm: định nghĩa model, hàm nghiệp vụ, và hàm main thực thi.

Hình 2.X. Cấu trúc file ví dụ quản lý danh mục sách

Phân tích chi tiết và giải thích

Định nghĩa Data Class Book

Khai báo model dữ liệu sách sử dụng Data Class:

```
data class Book(
    val id: Int,
    val title: String,
    val author: String,
    val category: String,
    var price: Double,
    var stock: Int
)
```

Giải thích: Từ khóa *data class* giúp trình biên dịch tự động sinh toString() để in thông tin sách, equals() để so sánh hai cuốn sách và copy() để tạo bản sao với giá khác nhau mà không cần viết thêm bất kỳ dòng code nào.

Hàm lọc xà xử lý danh sách

```
fun locSachTheoTheLoai(danhSach: List<Book>, theLoai: String):
  List<Book> {
    return danhSach.filter { it.category == theLoai }
}

fun tinhTongGiaTri(danhSach: List<Book>): Double {
    return danhSach.sumOf { it.price * it.stock }
}
```

Giải thích: Hàm *locSachTheoTheLoai* sử dụng hàm bậc cao *filter{}* – một trong những tính năng mạnh mẽ của lập trình hàm trong Kotlin. Biểu thức lambda *{ it.category == theLoai }* được truyền vào như một điều kiện lọc ngắn gọn, thay thế cho vòng lặp tường minh.

Hàm main – Thực thị và hiển thị kết quả

```
fun main() {
    val bookList = listOf(
        Book(1, "Kotlin in Action",    "JetBrains", "Lập
  trình", 299000.0, 15),
        Book(2, "Android Dev Guide",   "Google",   "Lập
  trình", 350000.0, 8),
        Book(3, "Doraemon Vol.1",      "Fujiko",   "Manga",
  89000.0,  30),
        Book(4, "Clean Architecture", "Robert",   "Lập trình",
  320000.0, 5)
    )

    val sachLapTrinh = locSachTheoTheLoai(bookList, "Lập
  trình")
    println("=== Sách lập trình ===")
    sachLapTrinh.forEach { println("  ${it.title} -
  ${it.price}đ") }

    val tongGiaTri = tinhTongGiaTri(bookList)
    println("Tổng giá trị kho sách: ${tongGiaTri}đ")
}
```

Kết quả minh họa

Khi thực thi chương trình, màn hình console xuất ra kết quả:

```
     === Sách lập trình ===
  Kotlin in Action - 299000.0đ
  Android Dev Guide - 350000.0đ
  Clean Architecture - 320000.0đ
Tổng giá trị kho sách: 9210000.0đ
```

Hình 2.X. Kết quả chạy ví dụ quản lý danh mục sách

Kết luận

Qua ví dụ trên, có thể rút ra một số nhận định chính:

- Data Class giúp định nghĩa model dữ liệu cực kỳ súc tích, tự động có các phương thức cần thiết mà không phải viết tay.
- Các hàm bậc cao như filter, forEach, sumOf giúp xử lý tập hợp dữ liệu ngắn gọn và biểu đạt hơn so với vòng lặp truyền thống.
- Cú pháp Kotlin nhất quán, rõ ràng, giúp đọc và hiểu code dễ dàng ngay cả khi chưa có nhiều kinh nghiệm.

Đồ án (Bổ sung sau)

Áp dụng kiến thức ngôn ngữ vào đồ án "Ứng dụng bán sách"

Sau khi nghiên cứu các khái niệm ngôn ngữ cơ bản của Kotlin (mục 2.1), đồ án "Ứng dụng bán sách" đã vận dụng trực tiếp các kiến thức này để xây dựng tầng Model – lớp nền tảng của toàn bộ hệ thống theo kiến trúc MVVM.

Định nghĩa các Data Class trong đồ án

Toàn bộ dữ liệu của ứng dụng được biểu diễn thông qua các Data Class rõ ràng, nhất quán. Mỗi lớp đại diện cho một thực thể nghiệp vụ trong hệ thống bán sách:

Model Book – Thực thể sách

```
data class Book(
    val id: String,
    val title: String,
    val author: String,
    val category: String,
    val price: Double,
    val imageUrl: String,
    val description: String,
    val rating: Float = 0f,
    val reviewCount: Int = 0
)
```

Đây là Model trung tâm của ứng dụng. Toàn bộ màn hình danh sách sách, chi tiết sách và giỏ hàng đều xoay quanh đối tượng Book. Tham số mặc định (rating = 0f, reviewCount = 0) giúp tạo đối tượng linh hoạt khi dữ liệu từ API chưa có đầy đủ thông tin đánh giá.

Hình 2.X. Cấu trục thư mục chứa các Data Class trong đồ án

Model CartItem – Thực thể giỏ hàng

```
data class CartItem(
    val book: Book,
    var quantity: Int
) {
    val totalPrice: Double get() = book.price * quantity
}
```

CartItem sử dụng thuộc tính tính toán (computed property) totalPrice – một tính năng ngôn ngữ của Kotlin cho phép định nghĩa thuộc tính không lưu trữ trực tiếp mà tính toán theo yêu cầu. Điều này đảm bảo tổng tiền luôn đồng bộ khi số lượng thay đổi mà không cần cập nhật thủ công.

Ứng dụng Control Flow trong xử lý nghiệp vụ

Biểu thức when được sử dụng rộng rãi trong đồ án để phân loại và xử lý các trạng thái khác nhau của dữ liệu:

```
fun layTenDanhMuc(category: String): String = when (category) {
    "tech"     -> "Công nghệ & Lập trình"
    "fiction"  -> "Văn học & Tiểu thuyết"
    "manga"    -> "Truyện tranh & Manga"
    "business" -> "Kinh doanh & Khởi nghiệp"
    else       -> "Khác"
}
```

Hình 2.X. Màn hình danh sách sách theo danh mục trong ứng dụng

Kết quả đạt được

Việc áp dụng các khái niệm ngôn ngữ Kotlin vào đồ án mang lại những kết quả cụ thể:

- Tầng Moodel được định nghĩa rõ ràng, ngắn gọn với Data Class – giảm đáng kể lượng boilerplate so với Java.

- Tham số mặc định và named parameter giúp khởi tạo đối tượng linh hoạt trong nhiều ngữ cảnh khác nhau.
- Computed property (get()) giúp đảm bảo tính nhất quán dữ liệu mà không cần logic cập nhật thủ công

CHƯƠNG 4: KOTLIN NÂNG CAO VÀ LẬP TRÌNH HÀM

Lý thuyết

Null Safety – An toàn với giá trị null

Một trong những nguyên nhân phổ biến nhất gây ra lỗi ứng dụng trong Java là NullPointerException (NPE) – lỗi xảy ra khi truy cập một đối tượng có giá trị null. Kotlin giải quyết vấn đề này một cách triệt để ngay tại giai đoạn biên dịch thông qua hệ thống kiểu phân biệt Nullable và Non-Nullable.

Phân biệt Nullable và Non-Nullable

Trong Kotlin, mọi kiểu dữ liệu mặc định đều là Non-Nullable – tức là không thể gán null. Để cho phép giá trị null, phải khai báo tường minh bằng dấu hỏi (?) sau kiểu dữ liệu:

```
var name: String = "Kotlin” //Non-nullable: không thể gán null
var author: String? = null    // Nullable: có thể là null
// name = null      // Lỗi biên dịch - trình biên dịch từ chối
```

Toán tử truy cập an toàn – Safe Call Operator (?.)

Khi làm việc với biến Nullable, toán tử ?. giúp truy cập thuộc tính hoặc gọi phương thức một cách an toàn mà không gây NPE. Nếu đối tượng là null, toàn bộ biểu thức trả về null thay vì ném ngoại lệ:

```
val author: String? = null
val length = author?.length   // Kết quả: null (không crash)
val upper  = author?.uppercase() // Kết quả: null
```

Toán tử Elvis – Elvis Operator (?:)

Toán tử ?: cung cấp một giá trị mặc định khi biểu thức bên trái là null:

```
val displayName = author ?: "Tác giả không rõ"
// Nếu author là null, displayName = "Tác giả không rõ"
```

Toán tử khẳng định null – Not-Null Assertion (!!)

Toán tử !! được dùng khi lập trình viên chắc chắn giá trị không phải null tại thời điểm thực thi. Nếu giá trị là null, NPE sẽ được ném ra. Do đó, toán tử này cần được dùng cẩn trọng:

```
val length = author!!.length  // Ném NPE nếu author là null
```

> *Khuyến nghị: Ưu tiên sử dụng safe call (?.) và Elvis (?:) thay vì (!!) để giữ mã nguồn an toàn. Toán tử !! chỉ dùng khi có đủ cơ sở logic đảm bảo giá trị không null.*

Extension Functions – Hàm mở rộng

Extension Functions (hàm mở rộng) là một trong những tính năng ấn tượng nhất của Kotlin. Chúng cho phép thêm hàm mới vào một lớp đã tồn tại – kể cả các lớp của thư viện hay lớp của hệ thống – mà không cần kế thừa hoặc sửa đổi mã nguồn gốc. Đây là giải pháp thay thế an toàn và linh hoạt cho pattern Decorator.

Cú pháp khai báo

```
fun TênLớp.tênHàmMới(thamSố: KiểuDữLiệu): KiểuTrảVề {
    // this ở đây là đối tượng của TênLớp
    return giá_trị
}
```

Ví dụ: *mở rộng lớp String để định dạng giá tiền:*

```
fun Double.toVND(): String {
    return "%,.0f VNĐ".format(this)
}


val price = 299000.0
println(price.toVND()) // Kết quả: 299,000 VNĐ
```

Giải thích: Hàm *toVND()* được thêm vào lớp *Double* sẵn có của Kotlin. Từ khóa this tham chiếu đến giá trị Double đang được gọi hàm. Nhờ Extension Function, lập trình viên có thể gọi hàm này như thể nó là một phương thức gốc của Double.

Higher-Order Functions và Lambda

Kotlin hỗ trợ lập trình hàm (Functional Programming) ở cấp ngôn ngữ. Hàm trong Kotlin là công dân hạng nhất (first-class citizen) – có nghĩa là hàm có thể được lưu vào biến, truyền làm tham số, và trả về từ hàm khác.

Hàm bậc cao (Higher-Order Fuctionns)

Hàm bậc cao là hàm nhận một hàm khác làm tham số hoặc trả về một hàm. Đây là nền tảng của lập trình hàm trong Kotlin:

```
fun apDungVoiDanhSach(
    danhSach: List<Book>,
    dieuKien: (Book) -> Boolean
): List<Book> {
    return danhSach.filter(dieuKien)
}
// Gọi hàm với lambda
val sachRe = apDungVoiDanhSach(books) { it.price < 200000.0 }
```

Các hàm xử lý tập hợp phổ biến

Kotlin cung cấp sẵn nhiều hàm bậc cao trong Collections API, giúp thao tác dữ liệu trở nên cực kỳ súc tích và biểu đạt:

- **filter {}** – Lọc phần tử thỏa điều kiện.
- **map {}** – Biến đổi mỗi phần tử thành dạng khác.
- **find {}** – Tìm phần tử đầu tiên thỏa điều kiện, trả về null nếu không tìm thấy.
- **sumOf {}** – Tính tổng giá trị số học của các phần tử.
- **groupBy {}** – Nhóm các phần tử theo một tiêu chí, trả về Map<K, List<T>>.

Ví dụ *kết hợp nhiều hàm xử lý tập hợp:*

```
val sachTheoGia = books
    .filter { it.stock > 0 }        // Chỉ lấy sách còn hàng
    .sortedBy { it.price }         // Sắp xếp tăng dần theo giá
    .map { "${it.title}: ${it.price}" }  // Lấy tên và giá
```

Sealed Class – Lớp niêm phong

Sealed Class (lớp niêm phong) là một loại lớp đặc biệt trong Kotlin, nơi tất cả các lớp con được khai báo tường minh trong cùng một file. Điều này giúp trình biên dịch biết chính xác tất cả các trạng thái có thể có của một kiểu dữ liệu, cho phép kiểm tra toàn diện (exhaustive) trong biểu thức when mà không cần nhánh else.

Ứng dụng điển hình nhất của Sealed Class trong phát triển Android là biểu diễn các trạng thái tải dữ liệu từ API (Loading, Success, Error):

```
sealed class UiState<out T> {
    object Loading                    : UiState<Nothing>()
    data class Success<T>(val data: T) : UiState<T>()
    data class Error(val message: String) : UiState<Nothing>()
}
```

Sử dụng Sealed Class cùng with biểu thức when:

```
when (uiState) {
    is UiState.Loading ->
        CircularProgressIndicator() // Hiện vòng xoay
    is UiState.Success ->
        BookList(books = uiState.data) // Hiện danh sách sách
    is UiState.Error ->
        ErrorMessage(text = uiState.message) // Hiện thông báo
   lỗi
}
```

> *Lưu ý: Khi dùng Sealed Class với when, nếu thiếu một nhánh xử lý, trình biên dịch sẽ cảnh báo lỗi ngay tại thời điểm biên dịch. Đây là cơ chế bảo vệ cực kỳ hữu ích, giúp đảm bảo không bỏ sót trạng thái nào trong vòng đời tải dữ liệu.*

Coroutines – Lập trình bất đồng bộ

Coroutines là một trong những tính năng đặc sắc và quan trọng nhất của Kotlin. Chúng cung cấp một cách tiếp cận lập trình bất đồng bộ (asynchronous programming) nhẹ nhàng, súc tích và không gây ra callback hell – vấn đề phổ biến khi lập trình với Thread hay các thư viện bất đồng bộ truyền thống.

Tại sao lập trình bắt đồng bộ?

Trong ứng dụng Android, giao diện người dùng chạy trên Main Thread. Nếu thực hiện các tác vụ tốn thời gian (như gọi API, đọc cơ sở dữ liệu) trực tiếp trên Main Thread, ứng dụng sẽ bị đóng băng (ANR – Application Not Responding). Coroutines giải quyết vấn đề này bằng cách cho phép tạm dừng và tiếp tục thực thi mà không chặn thread.

Các khái niệm cốt lõi của Coroutines

- **suspend fun:** – Từ khóa đánh dấu hàm có thể tạm dừng và tiếp tục mà không chặn thread. Chỉ được gọi từ trong một Coroutine hoặc suspend function khác.
- **CoroutinScope**: Định nghĩa phạm vi sống của Coroutine. Khi scope bị hủy, mọi Coroutine bên trong bị hủy theo, tránh Memory Leak.
- **Launch**: Khởi tạo một Coroutine "fire and forget" – Không chờ kêt quả trả về.
- **async/await:** Khởi tạo Coroutine và chờ kết quả trả về, dùng khi cần chạy song song nhiều tác vụ.

Dispatchers – Điều phối luồng thực thi

- **Dispachers.IO**: Dành cho tác vụ I/O như gọi API, đọc ghi cơ sở dữ liệu.
- **Dispatcher**. **Main:** Dành cho cập nhật giao diện người dùng (UI thread).
- **Dispatcher**. **Default:** Dành cho tác vụ tính toán nặng trên CPU.

Ví dụ gọi API bắt đồng bộ với Coroutines trong ViewModel:

```
fun loadBooks() {
    viewModelScope.launch {          // Phạm vi gắn với
  ViewModel
        _uiState.value = UiState.Loading
        try {
            val books = withContext(Dispatchers.IO) {
                bookRepository.getBooks() // Gọi API trên IO
  thread
            }
            _uiState.value = UiState.Success(books)
        } catch (e: Exception) {
```

```
            _uiState.value = UiState.Error(e.message ?: "Lỗi
   không xác định")
        }
    }
}
```

Ví dụ và giải thích

Ví dụ minh họa: Tải và xử lý dữ liệu sách bất đồng bộ

Mục tiêu của ví dụ

Ví dụ "Tải và xử lý dữ liệu sách bất đồng bộ" được xây dựng nhằm làm rõ hai khái niệm nâng cao quan trọng của Kotlin:

- **Null Safety:** Xử lý an toàn các giá trị có thể null từ phản hồi API.
- **Coroutines kết hợp Sealed Class:** Mô phỏng quy tình tải dữ liệu thực tế với các trạng thái Loading → Success/Error.
- **Extension Functions**: Định dạng và xử lý dữ liệu theo cách gọn gàng, có thể tái sử dụng.

Cấu trúc dự án

Ví dụ được tổ chức theo cấu trục phân lớp đơn giản nhằm phản ánh kiến trúc MVVM thực tế của đồ án:

Hình 3.X. Cấu trục thư mục ví dụ tải dữ liệu bất đồng bộ

Phân tích chi tiết và giải thích

Định nghĩa Sealed Class UiState

Sealed Class UiState đóng vai trò là "hợp đồng" giữa ViewModel và giao diện, biểu diễn rõ ràng ba trạng thái có thể xảy ra:

```
sealed class UiState<out T> {
    object Loading : UiState<Nothing>()
    data class Success<T>(val data: T) : UiState<T>()
```

```
    data class Error(val message: String) : UiState<Nothing>()
}
```

Giải thích: Từ khóa *out* T là Covariance (hiệp biến) – cho phép UiState<Book> được coi là UiState<Any>. Điều này giúp Sealed Class có thể tái sử dụng cho mọi loại dữ liệu (sách, đơn hàng, người dùng) mà không cần tạo class riêng biệt.

Extennsion Function xử lý dữ liệu

Extension Functions được dùng để bổ sung khả năng định dạng cho các kiểu dữ liệu hiện có:

```
fun Double.toFormattedPrice(): String {
    return "%,.0f VNĐ".format(this)
}


fun String?.orDefault(default: String = "Không rõ"): String {
    return this ?: default
}
```

Giải thích: Hàm *orDefault()* được khai báo trên kiểu String? (nullable String). Toán tử Elvis(?:) bên trong trả về giá trị mặc định khi chuỗi là null. Đây là ví dụ điển hình về cách kết hợp Null Safety và Extension Function để tạo ra API ngôn ngữ trực quan.

Repository mô phỏng gọi API với Coroutines

```
class BookRepository {
    suspend fun fetchBooks(): List<Book> {
        delay(1500L) // Mô phỏng độ trễ mạng
        return listOf(
            Book("1", "Kotlin in Action",  "JetBrains",
  299000.0, null),
            Book("2", "Android Dev Guide", null,
  350000.0, "Sách chính thức"),
        )
    }
}
```

Giải thích: Từ khóa *suspend* đánh dấu hàm này có thể tạm dừng và tiếp tục. Hàm *delay()* của Coroutines sẽ tạm dừng Coroutine đang chạy mà không chặn thread, khác hoàn toàn với *Thread.sleep()* trong Java – vốn sẽ chặn toàn bộ thread.

Kết quả minh họa

Khi ứng dụng chạy, luồng hoạt động diễn ra theo trình tự:

- Giao diện hiển thị CircularProgressIndicator (trạng thái Loading).
- Sau 1.5 giây, dữ liệu được tải xong, giao diện chuyển sang hiển thị danh sách sách.
- Nếu xảy ra lỗi (mất kết nối mạng), giao diện chuyển sang hiển thị thông báo lỗi.

Hình 3.X. Các trạng thái giao diện tương ứng vơi UiState.Loading, Success và Error
Hình 3.X. Các trạng thái giao diện tương ứng vơi UiState.Loading, Success và Error
Hình 3.X. Các trạng thái giao diện tương ứng vơi UiState.Loading, Success và Error

Kết luận

Ví dụ "Tải và xử lý dữ liệu sách bất đồng bộ" minh họa cách Kotlin Nâng cao giải quyết các bài toán thực tế trong phát triển Android:

- Null Safety loại bỏ hoàn toàn nguy cơ NPE ngay tại giai đoạn biên dịch, đặc biệt quan trọng khi xử lý dữ liệu từ API có thể thiếu trường.
- Sealeed Class tạo ra hợp đồng trạng thai rõ ràng và an toàn, buộc lập trình viên phải xử lý mọi trường hợp có thể xảy ra.
- Coroutines làm cho mã xử lý bất đồng bộ trông và đọc giống mã đồng bộ thông thường, giảm đáng kể độ phức tập nhận thức.

Đồ án (Bổ sung sau)

Tổng quan áp dụng Kotlin Nâng cao trong đồ án

Đồ án "Ứng dụng bán sách" vận dụng toàn diện các tính năng Kotlin ngân cao đã được nghiên cứu để xây dựng tầng Repository và ViewModel – hai tầng cốt lõi trong kiến trúc MVVM của ứng dụng Android hiện đại

Sealed Class quản lý trạng thái giao diện

Toàn bộ luồng tải dữ liệu trong đồ án được quản lý thống nhất thông qua Sealed Class UiState. Đây là giải pháp tập trung và rõ ràng để biểu diễn các trạng thái Loading, Success và Error cho từng màn hình:

```
// Áp dụng trong BookListViewModel
private val _booksState =
   MutableStateFlow<UiState<List<Book>>>(UiState.Loading)
val booksState: StateFlow<UiState<List<Book>>> = _booksState


fun loadBooks(category: String? = null) {
    viewModelScope.launch {
        _booksState.value = UiState.Loading
        try {
            val result = withContext(Dispatchers.IO) {
                bookRepository.getBooks(category)
            }
            _booksState.value = UiState.Success(result)
        } catch (e: IOException) {
            _booksState.value = UiState.Error("Lỗi kết nối
   mạng. Vui lòng thử lại.")
        }
    }
}
```

Hình 3.X. Sơ đồ luồng trạng thái UiState trong đồ án bán sách

Null Safety trong xử lý dữ liệu API

Dữ liệu phản hồi từ các API bên ngoài thường có các trường tùy chọn (optional fields) có thể vắng mặt. Kotlin Null Safety kết hợp Extension Function giúp xử lý những trường hợp này một cách an toàn và nhất quán:

```
fun BookDto.toDomain(): Book {
    return Book(
```

```
        id          = this.id,
        title       = this.title.orDefault("Không có tiêu đề"),
        author      = this.author?.trim().orDefault(),
        category    = this.category ?: "Khác",
        price       = this.price ?: 0.0,
        imageUrl    = this.imageUrl.orEmpty(),
        description =
   this.description?.take(500).orDefault("Chưa có mô tả"),
        rating      = this.rating ?: 0f,
        reviewCount = this.reviewCount ?: 0
    )
}
```

Giải thích: Hàm toDomain() là Extension Function khai báo trên lớp BookDto (Data Transfer Object từ API), chuyển đổi dữ liệu thô từ API thành đối tượng Book trong domain. Toán tử Elvis (?:) và hàm orDefault() đảm bảo không có trường nào là null khi đi vào tầng domain.

Coroutines trong kiến trúc đồ án

Toàn bộ các tác vụ gọi mạng và đọc ghi cơ sở dữ liệu trong đồ án đều chạy trên Coroutines với Dispatcher phù hợp, đảm bảo Main Thread luôn tự do để cập nhật giao diện mượt mà

- viewModelScope.launch: Sử dụng trong ViewModel để khởi động Coroutine gắn với vòng đời ViewModel. Khi ViewModel bị hủy, tất cả Coroutine đang chạy cũng bỉ hủy tự động
- withContext(Dispachers.IO): Chuyển thực thi sang IO Thread khi gọi API hoặc truy vấn Room Database, tránh chặn Main Thread.
- StateFlow: Kết hợp với Coroutines để phát (emit) trạng thái mới đến giao diện, thay thế LiveData với hiệu năng và tính năng tốt hơn trong môi trường Compose.

Hình 3.X. Sơ đồ kiến trúc Coroutines trong ứng dụng bán sách

Kết quả đạt được

Việc áp dụng Kotlin Nâng cao trong đồ án mang lãi những kết quả thực tiễn:

- **Null Safety:** Loại bỏ hoàn toàn nguy cơ NullPointerException trong toàn bộ tầng domain và data của ứng dụng.
- **Sealed Class:** Giao diện phản ánh đúng và đầy đủ mọi trạng thái dữ liệu – không có trạng thái trung gian không xác định.
- **Coroutines:** Ứng dụng phản hồi mượt mà, không bao giờ bị đóng băng khi tải dữ liệu từ server, ngay cả với kết nối chậm.
- **Extension Functions:** Tầng mapping DTO → Domain ngắn gọn, dễ đọc và hoàn toàn có thể kiểm thử đơn lập (unit test)

CHƯƠNG 5: LẬP TRÌNH GIAO DIỆN VỚI JETPACK COMPOSE

Lý thuyết về Jetpack Compose

Jetpack Compose là gì?

Jetpack Compose là bộ công cụ hiện đại của Google dùng để xây dựng giao diện người dùng (UI) cho Android một cách khai báo (declarative) . Thay vì sử dụng XML để định nghĩa giao diện và tách biệt hoàn toàn với logic xử lý như cách truyền thống, Compose cho phép bạn xây dựng UI trực tiếp bằng mã Kotlin.

Điều này có nghĩa là toàn bộ giao diện của bạn được định nghĩa bằng các hàm Kotlin, giúp code trở nên trực quan, dễ đọc, dễ bảo trì và giảm thiểu lỗi hơn rất nhiều.

Nguyên lý hoạt động

Cốt lõi của Jetpack Compose nằm ở các hàm Composable . Một hàm Composable là một hàm Kotlin thông thường nhưng được đánh dấu bằng annotation @Composable. Annotation này báo cho trình biên dịch Compose biết rằng hàm này sẽ chuyển đổi dữ liệu thành các thành phần giao diện.~

- **Khai báo (Declarative):** Bạn khai báo giao diện sẽ trông như thế nào dựa trên trạng thái dữ liệu hiện tại. Bạn không cần viết các bước phức tạp để tìm kiếm view và cập nhật chúng như **findViewById** hay **setText**.
- **Tự động cập nhật**: Khi dữ liệu (state) thay đổi, Compose tự động **gọi lại** (recompose) các hàm **Composable** có liên quan để cập nhật giao diện. Cơ chế này được gọi là **recomposition.**

- **Minh hoạ:**

```
@Composable
fun Greeting(name: String) {
        Text(text = "Hello $name!")
}
```

Trong ví dụ trên, **Greeting** là một hàm Composable nhận tham số **name** và hiển thị nó. Nếu giá trị name thay đổi, Compose sẽ tự động chạy lại hàm **Greeting** để hiển thị tên mới.

Các khái niệm cơ bản

Để bắt đầu xây dựng giao diện với Compose, bạn cần nắm các khái niệm nền tảng sau:

Hàm Composable (@Composable)

Đây là đơn vị cơ bản nhất để xây dựng UI. Mọi thứ ta nhìn thấy trên màn hình đều được tạo từ các hàm này. Ta có thể tạo các composable của riêng mình và kết hợp chúng lại với nhau để tạo thành các composable phức tạp hơn.

Modifier

Modifier là một đối tượng bất biến giúp ta trang trí hoặc thay đổi hành vi của một composable . Nó cho phép ta căn chỉnh, thêm padding, thay đổi kích thước, xử lý sự kiện click, v.v. Modifier được sử dụng theo phong cách **chuỗi (chaining)** để áp dụng nhiều hiệu ứng cùng lúc.

```
@Composable
fun MyComponent() {
    Text(
        text = "Xin chào",
        modifier = Modifier
            .padding(16.dp)
            .fillMaxWidth()
            .clickable { /* Xử lý click */ }
    )
}
```

State

State là bất kỳ giá trị nào có thể thay đổi theo thời gian, ví dụ như nội dung trong ô nhập liệu, vị trí của thanh trượt, hay kết quả trả về từ mạng . Để Compose có thể theo dõi và cập nhật UI khi state thay đổi, ta cần lưu trữ state trong một đối tượng mutableStateOf và sử dụng remember để giữ giá trị này qua các lần recomposition.

```
@Composable
fun MyScreen() {
    var count by remember { mutableStateOf(0) }

    Button(onClick = { count++ }) {
      Text("Đã nhấn $count lần")
    }
}
```

Surface và MarterialTheme

Surface: Là một composable trung tâm trong Material Design, đại diện cho một mặt phẳng. Nó được dùng để làm nền, tạo bóng đổ, xử lý hình dạng (shape) và là nơi bắt đầu cho nhiều composable khác.

MarterialTheme: Cung cấp hệ thống màu sắc, kiểu chữ (typography) và hình dạng (shapes) nhất quán cho toàn bộ ứng dụng theo chuẩn Material Design 3.

Các thành phần giao diện chính (UI Components)

Layouts (Bố cuc)

Các Composable Layout giúp sắp xếp các thành phần UI trên màn hình.

Composable	Mục đích	Ví dụ sử dụng
Column	Sắp xếp các thành phần con theo chiều dọc (từ trên xuống dưới)	Một form đăng nhập với các trường nhập liệu xếp chồng
Row	Sắp xếp các thành phần con theo chiều ngang (từ trái sang phải)	Một hàng các biểu tượng hoặc nút bấm
Box	Xếp chồng các thành phần con lên nhau. Thành phần được khai báo sau sẽ nằm trên thành phần khai báo trước	Một hình ảnh nền với một lớp phủ (overlay) văn bản bên trên
LazyColumn / LazyRow	Phiên bản tương đương của RecyclerView, hiển thị danh sách các thành phần có thể cuộn một cách hiệu quả. Nó chỉ render những thành phần đang hiển thị trên màn hình	Danh sách hàng trăm bài báo, tin nhắn
LazyVerticalGrid	Hiển thị danh sách các thành phần dưới dạng lưới (grid) có thể cuộn	Thư viện ảnh, danh sách sản phẩm

Marterial Components (Thành phần Marterial)

Đây là các composable được Google thiết kế sẵn để tuân thủ các nguyên tắc của Material Design, giúp ứng dụng của bạn có giao diện chuyên nghiệp và nhất quán.

Loại thành phần	Các composable phổ biến	Mô tả ngắn
Tương tác cơ bản	Button, IconButton, FloatingActionButton, Checkbox, RadioButton, Switch, Slider	Cho phép người dùng thực hiện các hành động và lựa chọn
Hiển thị thông tin	Text, Image, Card, ProgressIndicator, Snackbar	Dùng để hiển thị văn bản, hình ảnh, nhóm nội dung, trạng thái tải và thông báo ngắn
Nhập liệu	TextField, OutlinedTextField	Cho phép người dùng nhập văn bản
Điều hướng & Cấu trúc	TopAppBar, BottomNavigation, NavigationDrawer, ModalDrawer, Scaffold, TabRow	Cung cấp cấu trúc tổng thể cho màn hình và hỗ trợ điều hướng giữa các phần. Scaffold là composable đặc biệt giúp bạn dễ dàng kết hợp các thành phần này lại với nhau
Tạm thời / Nổi bật	AlertDialog, Dialog, BottomSheet, Menu, Tooltip	Hiển thị các nội dung tạm thời, yêu cầu xác nhận hoặc cung cấp thêm tùy chọn

Ví dụ và giải thích (Bổ sung thêm ví dụ sau)

Ví dụ về Bộ đếm đơn giản (Counter)

Mục tiêu: Minh họa cách sử dụng **state** và **recomposition** trong Compose. Người dùng có thể nhấn nút để tăng số đếm và giao diện tự động cập nhật.

```
@Composable
fun Counter() {
        // 1. Khai báo state
    var count by remember { mutableStateOf(0) }

        // 2. Giao diện
        Column(
          horizontalAlignment = Alignment.CenterHorizontally,
          verticalArrangement = Arrangement.Center,
          modifier = Modifier.fillMaxSize()
        ) {
          // 3. Hiển thị giá trị
          Text(
            text = "Bạn đã nhấn $count lần",
            style = MaterialTheme.typography.headlineMedium
          )

          // 4. Nút bấm
          Button(
            onClick = { count++ },
            modifier = Modifier.padding(16.dp)
          ) {
            Text("Nhấn vào đây")
          }
    }
}
```

- **mutableStateOf(0)**: tạo một đối tượng MutableState<Int> với giá trị khởi tạo là 0. Đối tượng này có thể quan sát được – khi giá trị thay đổi, Compose sẽ biết và lên lịch recompose cho các composable đang đọc nó.
- **remember**: giúp lưu trữ đối tượng state qua các lần recompose. Nếu không có remember, mỗi lần recompose, mutableStateOf(0) sẽ được tạo lại, dẫn đến mất giá trị hiện tại.
- **by** là từ khóa delegate, cho phép đọc/ghi trực tiếp biến count thay vì phải dùng count.value. Nhờ đó, ta có thể viết count++ thay vì count.value++.
- **Column:** layout sắp xếp các phần tử con theo chiều dọc.

- **horizontalAlignment = Alignment.CenterHorizontally**: căn giữa các phần tử con theo chiều ngang.
- **verticalArrangement = Arrangement.Center:** phân bố các phần tử con sao cho chúng nằm giữa theo chiều dọc (tức là toàn bộ nội dung được căn giữa trong màn hình).
- **modifier = Modifier.fillMaxSize():** yêu cầu Column chiếm toàn bộ không gian có sẵn (cả chiều rộng và chiều cao).
- **text:** sử dụng template string "Bạn đã nhấn $count lần". Biến count được tự động theo dõi, khi count thay đổi, Text sẽ được recompose để hiển thị giá trị mới.
- **style = MaterialTheme.typography.headlineMedium:** áp dụng kiểu chữ headlineMedium từ theme Material, giúp văn bản có kích thước lớn và đậm phù hợp làm tiêu đề.
- **onClick = { count++ }:** hàm lambda được gọi mỗi khi người dùng nhấn nút. Ở đây, ta tăng giá trị count lên 1. Khi count thay đổi, state được cập nhật, kích hoạt recompose cho toàn bộ composable Counter.
- **modifier = Modifier.padding(16.dp):** thêm khoảng đệm 16.dp xung quanh nút, tạo khoảng cách với các thành phần khác.

Đồ án (Bổ sung sau)

CHƯƠNG 5: KIẾN TRÚC MVVM VÀ RETROFIT

5.1. Lý thuyết

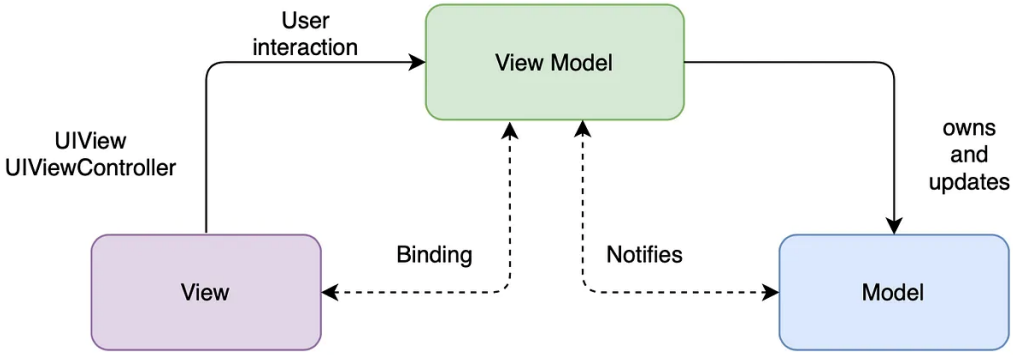
5.1.1. Giới thiệu tổng quan về mô hình MVVM

Mô hình MVVM là gì?

- Mô hình MVVM là một mô hình định nghĩa cấu trúc ứng dụng, được phát triển dựa trên kiến trúc của MVP. Mô hình này cho phép người dùng tách biệt giữa Model (dữ liệu), ViewModel (mã thực thi) và View (giao diện người dùng).

- Khác với các mô hình truyền thống, mô hình MVVM không cho phép người dùng xử lý sự kiện Click và các hoạt động khác bằng cách viết mã trực tiếp trên met Button. Nguyên nhân là vì trong MVVM, các control như ListView, Button hay SearchBar,… không thể kết nối với dữ liệu một cách trực tiếp. Thay vào đó, bạn sẽ phải sử dụng thuộc tính Command (thuộc kiểu ICommand) để kết nối các hành động với ViewModel.

MVVM có thể được hiểu như thế nào?



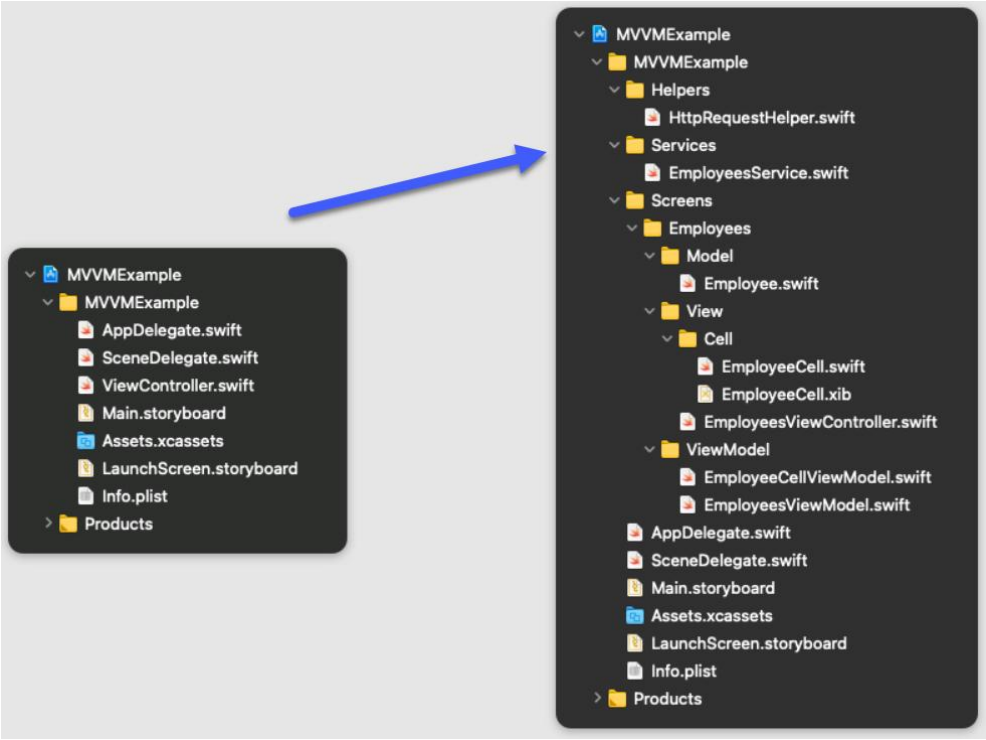
- **View**
  - View được hiểu là thành phần giao diện mô tả dữ liệu của một ứng dụng, đồng thời đây cũng là thành phần duy nhất cho phép người dùng tương tác trong chương trình.
  - View trong MVVM được đánh giá là tích cực hơn so với các mô hình khác nhờ vào khả năng thực hiện hành vi và cung cấp phản hồi cho người dùng với một số tính năng nổi bật như Command, Binding,

- **Model**: Giống với mô hình MVC, Model là đối tượng cho phép bạn truy xuất dữ liệu và thao tác trên dữ liệu thật sự.
- **ViewModel** : ViewModel đóng vai trò trung gian và có nhiệm vụ đồng bộ hóa dữ liệu khi truyền tải từ Model lên View, đồng thời xử lý các hoạt động từ View để cập nhật Model. View sẽ được ánh xạ (binding) tới ViewModel, nhưng ViewModel lại không biết thông tin của View mà thông tin này sẽ được ẩn giấu bởi Data-binding cùng cơ chế hoạt động của mô hình Observer. Trong đó, một ViewModel có khả năng ánh xạ (binding) từ nhiều View.

Lưu ý: Điểm đặc biệt của mô hình MVVM là sự tách biệt giữa các tầng và tầng bên dưới sẽ không bị phụ thuộc vào tầng bên trên. Cụ thể, các tầng dưới sẽ không được biết thông tin của các tầng bên trên, chẳng hạn như ViewModel không biết gì về View cụ thể mà nó đang liên kết và một ViewModel có thể sử dụng cho nhiều View khác nhau. Để liên lạc với View, ViewModel cần sử dụng Observer design pattern (hay binding data) với một hoặc hai chiều tùy theo nhu cầu riêng biệt của từng ứng dụng.

Cấu trúc thư mục trong MVVM

Mô hình MVVM thường bao gồm 3 thư mục chính, mỗi thư mục sẽ chứa những file code liên quan khác nhau, cụ thể là:



- **Views**
  - Thư mục View chứa các file giao diện và mỗi file giao diện sẽ đi kèm với một code-behind. Tuy nhiên, chúng ta thường sẽ không sử dụng file code-behind mà chuyển xuống class ViewModel.
  - Bạn cũng có thể sử dụng file code-behind, nhưng điều này thường không được khuyến khích vì nó có thể phá vỡ đi quy ước của mô hình MVVM. Trong file XAML, bạn có thể khai báo thuộc tính Datacontext hoặc sử dụng các thiết lập khác để liên kết ViewModel với giao diện người dùng. Tuy nhiên, cần lưu ý là bạn nên hạn chế code tại đây.
  - Trong mô hình MVVM, View với nhiệm vụ hiển thị giao diện người dùng và tạo nên sự chia tách gọn gàng giữa UI với Presentation Logic và Data.

- **Models**: Thư mục Models bao gồm các class chứa data và các liên kết validation, logic nghiệp vụ với mục đích đảm bảo tính toàn vẹn của data. Từ đó, bạn có thể dễ dàng tách chúng ta thư mục Repositories khác và sử dụng như một phần của mô hình MVVM.
- **ViewModels**:
    - Trong mô hình MVVM thường có thư mục riêng được tạo ra để chứa các lớp ViewModel tương ứng với mỗi file giao diện riêng biệt – đó là thư mục ViewModels. Thư mục này có nhiệm vụ quản lý và tổ chức logic của ViewModel trong các ứng dụng.
    - ViewModels có thể sử dụng các Model để định nghĩa dữ liệu và logic liên quan. ViewModel được ví như một trung gian đứng giữa View và Model, có nhiệm vụ gửi và nhận dữ liệu, từ đó cung cấp các khái niệm về DataContext, Binding, Behaviors SDK và cho phép người tách code-behind từ View đưa xuống ViewModel.
    - Bên cạnh đó, một lớp ViewModels còn chứa các logic hiển thị và state của ứng dụng. Như vậy, ViewModels sẽ chịu trách nhiệm cho các chức năng của ứng dụng và nó định nghĩa cho các thuộc tính (properties), commands và events để tương tác với các thành phần giao diện trong View, từ đó chuyển đổi controls trong view cần data-bind.

- **Data Binding**
    - Data Binding là một kỹ thuật quan trọng, được dùng để liên kết giữa giao diện UI và dữ liệu thông qua Business logic. Cụ thể, UI sẽ tự động cập nhật để hiển thị các thay đổi dữ liệu nhờ vào quá trình Data Binding. Không chỉ thế, trong WPF, Data Binding còn có khả năng hỗ trợ các chiều khác nhau – tức là cập nhật các thay đổi từ UI vào dữ liệu.
    - Kỹ thuật Data Binding trong MVVM được ví như một bước tiến mới mà bất kỳ lập trình viên nào cũng mong muốn khám phá. Thậm chí, Data Binding còn trở thành thành phần cốt lõi tạo nên cơ chế hoạt động của WPF. Qua đó, bạn có thể binding dữ liệu của bất kỳ đối tượng nào, từ các control đơn giản cho đến các user control phức tạp nhất. Đặc biệt, kỹ thuật này còn hỗ trợ người dùng thực hiện các công việc trên một cách dễ dàng mà không cần phải sử dụng đến bất cứ dòng code-behind nào.
- **Data Template**
    - Data Template là kỹ thuật được sử dụng cho các Control để tạo khuôn mẫu giao diện cho ứng dụng. Trong WPF, template có vai trò xác định cách thức hoặc cấu trúc của dữ liệu hay control khi hiển thị ra màn hình.
    - Data Template có khả năng gắn các dữ liệu dạng non-visuel vào một cấu trúc với một hoặc nhiều thành phần được hiển thị. Qua đó, dữ liệu sẽ được hiển thị theo đúng ý muốn của bạn. Ngoài ra, tính năng này cũng không sử dụng đến code-behind của ứng dụng giống như Data Binding.

- **Command**
    - Nếu như Data Bingding và Data Template đóng vai trò quan trọng trong việc hỗ trợ người dùng nhận biết các thành phần của dữ liệu và cập nhật dữ liệu đó, thì Command sẽ đảm nhận nhiệm vụ nhận dữ liệu tương tác người dùng và xử lý yêu cầu đó.
    - Cụ thể, các command sẽ xem dữ liệu và cung cấp cho người dùng bằng chức năng binding. Thông qua đó, command binding trong mô hình MVVM cho phép bạn xác định các phương pháp xử lý để kích hoạt thông qua các thao tác như phím tắt, chuột,…

5.1.2. Retrofit

Retrofit là gì?

Được phát triển bởi Square

Retrofit là một HTTP client type-safe cho Android và Java. Retrofit giúp dễ dàng kết nối đến một dịch vụ REST trên web bằng cách chyển đổi API thành Java Interface.

Retrofit rất mạnh mẽ giúp bạn dễ dàng xử lý dữ liệu JSON hoặc XML sau đó phân tích cú pháp thành Plain Old Java Objects (POJOs). Tất cả các yêu cầu GET, POST, PUT, PATCH, và DELETE đều có thể được thực thi.

Giống như hầu hết các phần mềm mã nguồn mở khác, Retrofit được xây dựng dựa trên một số thư viện mạnh mẽ và công cụ khác. Đằng sau nó, Retrofit làm cho việc sử dụng OkHttp (từ cùng một nhà phát triển) để xử lý các yêu cầu trên mạng. Ngoài ra, Retrofit không tích hợp bất kỳ một bộ chuyển đổi JSON nào để phân tích từ JSON thành các đối tượng Java. Thay vào đó nó đi kèm với các thư viện chuyển đổi JSON sau đây để xử lý điều đó

Cách dùng Retrofit

Để làm việc với Retrofit bạn cần triển khai cơ bản 3 lớp:

- Model class để ánh xạ với JSON data.
- Một interface dùng để định nghĩa các hàm và phương thức HTTP
- Retrofit.Builder Lớp để định nghĩa URL Endpoint cho các hoạt động liên quan tới HTTP

Request method

Mỗi method phải có một HTTP annotation. Có 5 annotations GET, POST, PUT, DELETE, and HEAD Bên trong mỗi annotation là một đoạn của URL.

```
@GET("users/list")
```

Bạn cũng có thể chỉ định tham số truy vấn trong URL

```
@GET("users/list?sort=desc")
```

URL MANIPULATION

```
interface GitHubService {
    @GET("users/{user}/repos")
    fun listRepos(@Path("user") user: String): Call<List<Repo>>
}
```

Khối {user] trong đoạn trên được thay thế bới tham số user trong hàm listRepos. Bằng việc sử dụng anotation @Path và truyền vào cùng chuỗi string trong khối {}

Tham số query cũng có thể add thêm được như sau:

```
@GET("group/{id}/users")
fun groupList(
    @Path("id") groupId: Int,
    @Query("sort") sort: String
): Call<List<User>>
```

Nếu mà tham số query phức tạp có thể sử dụng Map:

```
@GET("group/{id}/users")
fun groupList(
    @Path("id") groupId: Int,
    @QueryMap options: Map<String, String>
): Call<List<User>>
```

Requets Body

Một đối tượng có thể được chỉ định cho mục đích sử dụng làm phần body cho truy vấn với @Body annotation

```
@POST("users/new")
fun createUser(@Body user: User): Call<User>
```

Đối tượng cũng sẽ được chuyển đổi bằng cách sử dụng trình chuyển đổi được chỉ định trong Retrofit instance. Nếu không có trình chuyển đổi nào được thêm vào, chỉ có thể sử dụng RequestBody.

Form Encoded and multipart

Các phương thức cũng có thể được khai báo để gửi dữ liệu được mã hóa theo mẫu và nhiều phần. Dữ liệu được mã hóa biểu mẫu được gửi khi có annotation @FormUrlEncoding . Mỗi cặp key-value được chú thích bằng @Field chứa tên và đối tượng cung cấp giá trị.

```
@FormUrlEncoded
@POST("user/edit")
fun updateUser(
    @Field("first_name") first: String,
    @Field("last_name") last: String
): Call<User>
```

Yêu cầu nhiều phần được sử dụng với @Multipart .Mỗi phần được khai báo bằng cách sử dụng chú thích @Part.

```
@Multipart
@PUT("user/photo")
fun updateUser(
    @Part("photo") photo: RequestBody,
    @Part("description") description: RequestBody
): Call<User>
```

Header manipulation

Bạn có thể Header tĩnh cho phương thức sử dụng @Headers annotation.

```
@Headers("Cache-Control: max-age=640000")
@GET("widget/list")
fun widgetList(): Call<List<Widget>>
```

```
@Headers(
    "Accept: application/vnd.github.v3.full+json",
    "User-Agent: Retrofit-Sample-App"
)
@GET("users/{username}")
fun getUser(@Path("username") username: String): Call<User>
```

Lưu ý: các headers không ghi đè lên nhau. Tất cả các tiêu đề có cùng tên sẽ được bao gồm trong yêu cầu. Một tiêu đề yêu cầu có thể được cập nhật động bằng cách sử dụng chú thích @Header. Một tham số tương ứng phải được cung cấp cho @Header. Nếu giá trị là null, header sẽ bị bỏ qua. Nếu không, toString sẽ được gọi trên giá trị và kết quả được sử dụng.

```
@GET("user")
fun getUser(@Header("Authorization") authorization: String): Call<User>
```

Tương tự như các tham số truy vấn, đối với hearder phức tạp, có thể sự dụng Map

```
@GET("user")
fun getUser(@HeaderMap headers: Map<String, String>): Call<User>
```

Header cần thêm vào mỗi request tuy nhiên có thể sử dụng OkHttp Interceptor để thay thế.

Converters

Theo mặc định, Retrofit chỉ có thể giải tuần tự hóa (deserialize) các HTTPbodies thành ResponseBody của OkHttp và nó chỉ có thể chấp nhận loại RequestBody của nó cho @Body. Bộ chuyển đổi có thể được thêm vào để hỗ trợ các loại khác. Dưới đây là 6 thư viện phổ biến sử dụng.

- Gson: com.squareup.retrofit:converter-gson
- Jackson: com.squareup.retrofit:converter-jackson
- Moshi: com.squareup.retrofit:converter-moshi
- Protobuf: com.squareup.retrofit2:converter-protobuf
- Wire: com.squareup.retrofit2:converter-wire Và đối với XML, Retrofit hỗ trợ
- Simple Framework: com.squareup.retrofit2:converter-simpleframework

Caching là gì?

Caching là cách lưu trữ tạm thời dữ liệu được tìm nạp từ mạng trên bộ lưu trữ của thiết bị, để chúng ta có thể truy cập vào lần sau khi thiết bị ngoại tuyến hoặc nếu chúng ta muốn truy cập lại cùng một dữ liệu.

Lợi ích của việc Caching

- Giảm tiêu thụ băng thông.
- Tiết kiệm cho bạn thời gian bạn dành thời gian chờ đợi máy chủ cung cấp cho bạn phản hồi mạng.
- Tiết kiệm cho máy chủ gánh nặng của lưu lượng bổ sung.
- Nếu bạn cần truy cập lại cùng một tài nguyên mạng sau khi đã truy cập gần đây, thiết bị của bạn đã giành được Yêu cầu phải gửi yêu cầu đến máy chủ; Thay vào đó, nó sẽ nhận được phản hồi lưu trữ. Vì vậy, chuyển sang phần thực hành nào

Tạo instance Retrofit

Khi sử dụng Retrofit, cùng vs GSON, thì Retrofit instance sẽ có dạng thế này:

```
private val sRetrofit: Retrofit by lazy {
    Retrofit.Builder()
        .baseUrl(BASE_URL)
        .addCallAdapterFactory(RxJava2CallAdapterFactory.create())
        .addConverterFactory(GsonConverterFactory.create())
        .build()
}
```

Với instance ở trên có nghĩa là ta đã dùng OkHttpClient mặc định để thực thi requests. Điều đó không được "thân thiện" với bộ nhớ cho lắm . Chúng ta sẽ tạo ra instance của OkHttpClient để cache dữ liệu & xử lí dữ việc lấy dữ liệu một cách thuận tiện khi:

- Thiết bị đang offline.
- Thiết bị cần truy cập cùng một dữ liệu từ internet trong một khoảng thời gian ngắn.

Tạo instance Retrofit

Tạo phương thức kiểm tra kết nối internet

```
fun isNetworkAvailable(context: Context): Boolean {
        val connectivityManager =
    context.getSystemService(Context.CONNECTIVITY_SERVICE) as
    ConnectivityManager
        val network = connectivityManager.activeNetwork = return false
        val capabilities =
    connectivityManager.getNetworkCapabilities(network) ?: return false


        return
    capabilities.hasCapability(NetworkCapabilities.NET_CAPABILITY_
    INTERNET)
}
```

Định nghĩa một số tham số có sử dụng

Dưới đây là một số tham số có sử dụng trong quá trình tạo OkHttpClient. Ta chủ yếu quan tâm đến CACHE_SIZE, TIME_CACHE_ONLINE, TIME_CACHE_OFFLINE:

- CACHE_SIZE: chỉ ra lượng cache sẽ lưu là 10MB. Chú ý rằng CACHE_SIZE phải là kiểu long.
- TIME_CACHE_ONLINE: nếu có internet sẽ lấy cache lưu 1 phút trước, nếu quá 1 phút sẽ không lấy, 'max-age' là thuộc tính phụ trách việc này.
- TIME_CACHE_OFFLINE: nếu không có internet sẽ lấy cache lưu 1 ngày trước đây, nếu quá thì không lấy, "max-stale" là thuộc tính phụ trách việc này, 'only-if-cached' là để không request mà chỉ lấy cache.

```
companion object {
    private const val CACHE_SIZE = (10 * 1024 * 1024).toLong() //
10 MB
    private const val READ_TIMEOUT = 5000
    private const val WRITE_TIMEOUT = 5000
    private const val CONNECT_TIMEOUT = 5000

    private var CACHE_CONTROL = "Cache-Control"

    private const val TIME_CACHE_ONLINE = "public, max-age =
60" // 1 minute
    private const val TIME_CACHE_OFFLINE = "public, only-if-
cached, max-stale = 86400" // 1 day
}
```

Tạo OkHttpClient

Tạo instance của OkHttpClient với các thông số đã khai báo bên trên

```
private fun initClient(context: Context): OkHttpClient {
    return OkHttpClient.Builder().apply {
        readTimeout(READ_TIMEOUT.toLong(),
TimeUnit.MILLISECONDS)
        writeTimeout(WRITE_TIMEOUT.toLong(),
TimeUnit.MILLISECONDS)
        connectTimeout(CONNECT_TIMEOUT.toLong(),
TimeUnit.MILLISECONDS)
        retryOnConnectionFailure(true)
        cache(Cache(context.cacheDir, CACHE_SIZE))

        addInterceptor { chain ->
            var request = chain.request()

            // Xử lý Header Cache dựa trên trạng thái mạng
            val cacheHeaderValue = if (isNetworkAvailable(context)) {
                TIME_CACHE_ONLINE
            } else {
                TIME_CACHE_OFFLINE
            }

            // Thêm API Key vào URL Query Parameters
            val httpUrl = request.url.newBuilder()
                .addQueryParameter(QUERRY_PARAMETER_API_KEY,
API_KEY)
                .build()
```

```
            request = request.newBuilder()
                .header(CACHE_CONTROL, cacheHeaderValue)
                .url(httpUrl)
                .build()

            chain.proceed(request)
        }
    }.build()
}
```

Thêm instance OkHttpClient vừa tạo vào Retrofit

```
companion object {
    @Volatile
    private var sRetrofit: Retrofit? = null

    fun getInstance(context: Context): Retrofit {
        return sRetrofit ?: synchronized(this) {
            sRetrofit ?: Retrofit.Builder()
                .baseUrl(BASE_URL)
                // Gọi hàm initClient đã chuyển đổi ở bước trước
                .client(initClient(context))
                .addCallAdapterFactory(RxJava2CallAdapterFactory.create())
                .addConverterFactory(GsonConverterFactory.create())
                .build()
                .also { sRetrofit = it }
        }
    }
}
```

5.2. Ví dụ và giải thích

Tầng Model: Định nghĩa cấu trúc dữ liệu

Đoạn code này định nghĩa "khuôn mẫu" cho dữ liệu mà ứng dụng sẽ nhận về.

```
data class Post(
    val id: Int,
    val title: String,
    val body: String
)
```

Tầng Network: Interface định nghĩa API

Đây là nơi bạn ra lệnh cho Retrofit biết cần phải làm gì với Server.

```
interface PostApiService {
    @GET("posts")
    suspend fun getPosts(): List<Post>
}
```

Giải thích chi tiết:

- **@GET("posts")**: Đây là một Annotation. Nó nói với Retrofit rằng: "Hãy gửi một yêu cầu HTTP loại GET tới đường dẫn /posts".
- **suspend**: Từ khóa cực kỳ quan trọng trong Kotlin Coroutines. Nó đánh dấu hàm này là "hàm tạm dừng", nghĩa là nó có thể chạy các tác vụ mạng tốn thời gian mà không làm đứng (treo) giao diện người dùng.
- **List<Post>**: Kiểu dữ liệu trả về. Retrofit sẽ tự động hiểu rằng nó cần trả về một danh sách các đối tượng Post mà chúng ta đã định nghĩa ở trên.

Tầng Network: Khởi tạo Retrofit Instance

Đoạn code này thiết lập cấu hình kết nối mạng dùng chung cho toàn app.

```
object RetrofitInstance {
    private const val BASE_URL = "https://jsonplaceholder.typicode.com/"

    val api: PostApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(PostApiService::class.java)
    }
}
```

Giải thích chi tiết:

- **object**: Tạo ra một Singleton, nghĩa là trong suốt quá trình app chạy, chỉ có duy nhất một thực thể RetrofitInstance được tạo ra để tiết kiệm tài nguyên.
- **by lazy**: Cơ chế "khởi tạo chậm". Biến api sẽ chỉ được tạo ra khi lần đầu tiên có ai đó gọi đến nó, giúp app khởi động nhanh hơn.
- **baseUrl**: Địa chỉ gốc của Server. Các Endpoint sau này sẽ được nối đuôi vào đây.
- **GsonConverterFactory**: Đây là bộ "thông dịch viên". Nó tự động biến đổi chuỗi JSON (văn bản) từ Server thành đối tượng Kotlin (Object) một cách tự động.

Tầng ViewModel: Xử lý Logic và Quản lý Trạng thái

Đây là bộ não điều khiển sự tương tác giữa Dữ liệu và Giao diện.

```
class PostViewModel : ViewModel() {
    private val _posts = MutableStateFlow<List<Post>>(emptyList())
    val posts: StateFlow<List<Post>> = _posts

    private val _isLoading = MutableStateFlow(false)
    val isLoading: StateFlow<Boolean> = _isLoading

    fun fetchPosts() {
        viewModelScope.launch {
            _isLoading.value = true
            try {
                val response = RetrofitInstance.api.getPosts()
                _posts.value = response
            } catch (e: Exception) {
                // Handle error
            } finally {
                _isLoading.value = false
            }
        }
    }
}
```

Giải thích chi tiết:

- **MutableStateFlow vs StateFlow**: Đây là kỹ thuật Encapsulation (Đóng gói). Ta dùng Mutable (có thể sửa) bên trong ViewModel để cập nhật dữ liệu, nhưng chỉ cho phép View bên ngoài đọc dữ liệu thông qua StateFlow (chỉ đọc) để đảm bảo an toàn, tránh việc UI vô tình sửa đổi dữ liệu gốc.

- **viewModelScope.launch**: Tạo ra một môi trường (scope) để chạy Coroutine. Nếu người dùng thoát màn hình này, mọi tác vụ mạng đang chạy dở sẽ tự động bị hủy để tránh lãng phí pin và tài nguyên (Memory Leak).
- **try - catch - finally**: Cấu trúc xử lý lỗi mẫu mực. try để gọi mạng, catch để bắt lỗi nếu mất kết nối, và finally để tắt vòng xoay Loading dù thành công hay thất bại.

Tầng View: Hiển thị với Jetpack Compose

Đoạn code này dùng để vẽ giao diện dựa trên dữ liệu nhận được.

```
@Composable
fun PostScreen(viewModel: PostViewModel = viewModel()) {
    val posts by viewModel.posts.collectAsState()
    val isLoading by viewModel.isLoading.collectAsState()

    if (isLoading) {
        CircularProgressIndicator()
    } else {
        LazyColumn {
            items(posts) { post ->
                Text(text = post.title)
            }
        }
    }
}
```

Giải thích chi tiết:

- **collectAsState()**: Chuyển đổi dữ liệu từ luồng Flow của ViewModel thành một State mà Compose có thể hiểu được. Khi dữ liệu trong Flow thay đổi, hàm @Composable này sẽ tự động chạy lại (Recomposition) để cập nhật màn hình.
- **if (isLoading)**: Logic điều kiện trực tiếp trong UI. Nếu đang tải thì hiện vòng xoay (CircularProgressIndicator), nếu tải xong thì hiện danh sách.
- **LazyColumn**: Một thành phần cực kỳ tối ưu trong Compose (giống RecyclerView), nó chỉ vẽ những phần tử nào đang hiện trên màn hình, giúp app chạy mượt mà dù danh sách có hàng ngàn bài viết.

Tóm tắt luồng hoạt động:

- View gọi fetchPosts() của ViewModel.
- ViewModel bật isLoading = true và gọi Retrofit. Retrofit dùng HTTP GET lấy JSON từ Server về.
- Gson chuyển JSON thành danh sách Post.
- ViewModel nhận danh sách, tắt isLoading và đẩy vào posts (Flow).
- View thấy dữ liệu mới, tự động vẽ lại danh sách trên màn hình.

Kết quả:

5.3. Đồ án

CHƯƠNG 6: DEPENDENCY INJECTION VÀ HILT

6.1. Lý thuyết

6.1.1. Tổng quan về Dependency Injection

Dependency Injection (DI) là một mẫu thiết kế cho phép loại bỏ các phụ thuộc cứng và làm cho chúng có thể thay đổi, dù là tại thời điểm chạy (runtime) hay thời điểm biên dịch (compile time). DI là một dạng của Inversion of Control (IoC), một thuật ngữ rộng hơn bao gồm các kỹ thuật lập trình khác nhau nhằm tách rời các thành phần và cải thiện tính mô-đun.

Dependency Injection Là gì?

Ở cốt lõi của nó, DI là một kỹ thuật trong đó một đối tượng (gọi là "client") nhận được các phụ thuộc mà nó cần từ một đối tượng khác (gọi là "injector"). Điều này có nghĩa là, thay vì tạo ra các phụ thuộc bên trong chính nó, đối tượng client sẽ được cung cấp những phụ thuộc này từ bên ngoài.

Ví dụ: Trong một ứng dụng Android, bạn có thể cần một đối tượng Repository để truy xuất dữ liệu từ một cơ sở dữ liệu hoặc một dịch vụ web. Thay vì khởi tạo Repository trực tiếp bên trong Activity hoặc Fragment, bạn có thể sử dụng DI để cung cấp nó từ bên ngoài, giúp mã của bạn dễ dàng kiểm tra và bảo trì hơn.

Các Hình Thức Của Dependency Injection

- **Constructor Injection**: Các phụ thuộc được cung cấp thông qua constructor của đối tượng.
- **Setter Injection**: Các phụ thuộc được cung cấp thông qua các phương thức setter của đối tượng.
- **Interface Injection**: Đối tượng cung cấp các phương thức để nhận các phụ thuộc từ bên ngoài.

Lợi ích Của Dependency Injection

- **Tăng Tính Tái Sử Dụng Mã**: DI giúp tách rời các thành phần của ứng dụng, làm cho chúng dễ dàng tái sử dụng trong các bối cảnh khác nhau.
- **Dễ Dàng Kiểm Tra**: Với DI, bạn có thể dễ dàng thay thế các phụ thuộc thực bằng các phiên bản giả (mock) trong quá trình kiểm tra.

- **Giảm Sự Phụ Thuộc Cứng**: DI giúp giảm bớt sự phụ thuộc cứng, làm cho mã nguồn dễ dàng mở rộng và bảo trì.

Các Framework DI Tiêu Biểu

- **Dagger**

  Dagger là một framework DI mạnh mẽ và phổ biến được phát triển bởi **Google**. Nó sử dụng code generation để tạo ra các class DI, giúp cải thiện hiệu suất.

  Ưu Điểm Của Dagger:

  - Hiệu Suất Cao: Sử dụng code generation giúp Dagger nhanh và hiệu quả.
  - Tích Hợp Tốt Với Android: Được phát triển bởi Google, Dagger có sự tích hợp tốt với các thành phần của Android.
  - Hỗ Trợ Kiểm Tra: Dễ dàng tạo mock dependencies cho các bài kiểm tra.

  Ví Dụ Dagger:

```
// Định nghĩa một module cung cấp các phụ thuộc
@Module
class NetworkModule {
    @Provides
    fun provideRetrofit(): Retrofit {
        return Retrofit.Builder()
            .baseUrl("https://api.example.com")
            .build()
    }
}

// Component để tiêm các phụ thuộc
@Component(modules = [NetworkModule::class])
interface AppComponent {
    fun inject(activity: MainActivity)
}

// Sử dụng trong Activity
class MainActivity : AppCompatActivity() {
    @Inject lateinit var retrofit: Retrofit

    override fun onCreate(savedInstanceState: Bundle?) {
        (application as MyApplication).appComponent.inject(this)
        super.onCreate(savedInstanceState)
        // Sử dụng retrofit
    }
}
```

- **Koin**

Koin là một framework DI nhẹ và dễ sử dụng, đặc biệt phù hợp với Kotlin. Không giống như Dagger, Koin không sử dụng code generation mà dựa vào reflection, làm cho quá trình thiết lập nhanh chóng và dễ dàng hơn.

Ưu Điểm Của Koin:

  - Dễ Dàng Sử Dụng: Thiết lập nhanh chóng và cú pháp đơn giản.
  - Không Cần Code Generation: Không cần các annotation processors, giúp giảm thời gian biên dịch.
  - Tích Hợp Kotlin DSL: Sử dụng Kotlin DSL để định nghĩa các module, làm cho mã nguồn ngắn gọn và rõ ràng.

Ví Dụ Koin:

```
// Định nghĩa một module Koin
val appModule = module {
    single {
     Retrofit.Builder().baseUrl("https://api.example.com").build() }
}

// Khởi tạo Koin trong Application class
class MyApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        startKoin {
            androidContext(this@MyApplication)
            modules(appModule)
        }
    }
}

// Sử dụng trong Activity
class MainActivity : AppCompatActivity() {
    private val retrofit: Retrofit by inject()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Sử dụng retrofit
    }
}
```

- **Hilt**

Hilt là một framework DI được xây dựng trên nền tảng của Dagger và được Google phát triển để tối ưu hóa cho các ứng dụng Android. Hilt đơn giản hóa việc thiết lập và sử dụng DI bằng cách cung cấp các annotation chuyên dụng.

Ưu Điểm Của Hilt:

- Tích Hợp Tốt Với Android:: Được thiết kế để làm việc mượt mà với các thành phần của Android như Activity, Fragment, ViewModel, v.v.
- Thiết Lập Dễ Dàng: Cung cấp các annotation giúp giảm thiểu cấu hình phức tạp.
- Dựa Trên Dagger: Hưởng lợi từ hiệu suất cao của Dagger.

Ví dụ Hilt:

```
// Đánh dấu Application class với @HiltAndroidApp
@HiltAndroidApp
class MyApplication : Application()

// Định nghĩa một module Hilt
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {
    @Provides
    fun provideRetrofit(): Retrofit {
        return Retrofit.Builder()
            .baseUrl("https://api.example.com")
            .build()
    }
}

// Sử dụng Hilt trong Activity
@AndroidEntryPoint
class MainActivity : AppCompatActivity() {
    @Inject lateinit var retrofit: Retrofit

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Sử dụng retrofit
    }
}
```

6.1.2. Hilt

Vấn đề

Khi tạo các instances của các class trong project của bạn, bạn có thể thực hiện theo cách thủ công bằng cách thỏa mãn các phụ thuộc:

```
class FeedViewModel(
    private val loadCurrentMomentUseCase: LoadCurrentMomentUseCase
) : ViewModel()

//Ví dụ việc khởi tạo instance của class FeedViewModel:
FeedViewModel(LoadCurrentMomentUseCase())
```

và phụ thuộc bắc cầu mà class yêu cầu:

```
class LoadCurrentMomentUseCase(
    private val getTimeZoneUseCase: GetTimeZoneUseCase
) {...}

class FeedViewModel(
    private val loadCurrentMomentUseCase: LoadCurrentMomentUseCase
) : ViewModel()

//Ví dụ việc khởi tạo instance của class FeedViewModel:
FeedViewModel(LoadCurrentMomentUseCase(GetTimeZoneUseCase())
```

Việc khởi tạo instance một cách thủ công như trên các bạn có thể thấy nhược điểm rằng nếu constructor của class FeedViewModel tiếp tục cần truyền vào nhiều tham số hơn, các tham số có nhiều sự phụ thuộc hơn thì một điều kinh khủng sẽ xảy ra, các bạn cũng mường tượng được rồi đúng không Không những thế việc làm này còn lặp đi lặp lại từ các class khác. Một khối lượng duplicate code khủng khiếp. Tất nhiên các bạn vẫn có cách để tránh việc duplicate code trong trường hợp này nhưng tội gì phải làm vậy khi các bạn đã có Hilt.

Tất cả các ứng dụng sử dụng Hilt phải chứa một Application class có @HiltAndroidApp annotation vì nó kích hoạt generate code của Hilt tại thời điểm biên dịch:

```
@HiltAndroidApp
class MusicApp : Application()
```

và để Hilt có thể đưa các phụ thuộc vào một Activity thì Activity đó cần phải có @AndroidEntryPoint annotaion:

```
@AndroidEntryPoint
class PlayActivity : AppCompatActivity() { /* ... */ }
```

Để thêm một phụ thuộc, hãy sử dụng @Inject annotation vào các biến mà bạn muốn Hilt đưa vào. Tất cả các biến được đưa vào Hilt sẽ khả dụng khi super.onCreate được gọi:

```
@AndroidEntryPoint
class PlayActivity : AppCompatActivity() {

  @Inject lateinit var player: MusicPlayer

  override fun onCreate(savedInstanceState: Bundle) {
    super.onCreate(bundle)
    player.play("YHLQMDLG")
  }
}
```

Trong ví dụ này, đoạn code @Inject lateinit var player: MusicPlayer đang "tiêm" (inject) một instance có kiểu là MusicPlayer vào PlayActivity class. Nhưng làm thế nào Hilt biết cách cung cấp instance có kiểu là MusicPlayer? Hiện tại thì chưa, bạn cần cho Hilt biết cách làm điều đó bằng cách sử dụng **@Inject** annotation với constructor của MusicPlayer class:

```
class MusicPlayer @Inject constructor() {
  fun play(id: String) { ... }
}
```

Đây là tất cả những gì cần thiết để làm DI hoạt động trong project của bạn. Vừa rồi là một ví dụ đơn giản là MusicPlayer class không phụ thuộc vào bất kỳ class nào khác. Nhưng nếu bạn có các phụ thuộc khác được truyền dưới dạng các tham số, Hilt sẽ xử lý điều đó và đáp ứng các phụ thuộc đó khi cung cấp một instance của MusicPlayer:

```
//trong activity vẫn chỉ cần sử dụng
@Inject lateinit var player: MusicPlayer

class MusicPlayer @Inject constructor(
   private val musicRepository: MusicRepository
) {
  fun play(id: String) { ... }
}

class MusicRepository @Inject constructor() { ... }
```

Annotations recap

- Cho đến nay, chúng ta đã thấy rằng khi @Inject được sử dụng để chú thích cho constructor của một class, nó sẽ cho Hilt biết cách cung cấp các instance của class đó. Và khi nó chú thích một biến trong một @AndroidEntryPoint class, Hilt sẽ đưa một instance của kiểu đó vào class.
- @AndroidEntryPoint: có thể chú thích hầu hết các class của Android framework, không chỉ mỗi Activity. Nó sẽ tự tạo một class như một vùng chứa các phụ thuộc cho class có @AndroidEntryPoint annotation và tạo tất cả các biến có @Inject annotation với kiểu tướng ứng.
- @HiltAndroidApp: được sử dụng ở class kế thừa Application class. Ngoài việc kích hoạt việc generate code của Hilt nó còn tạo một vùng chứa các phụ thuộc được liến kết với Application class của bạn.

Hilt Modules

- Hilt ModulesTrong ví dụ gần nhất, MusicPlayer class có thêm sự phụ thuộc là MusicRepository class, thông thường việc giao tiếp giữa các lớp với nhau sẽ thông qua interface, nếu bạn truyền tham số cho constructor của MusicPlayer class là một interface hoặc một class nhưng bạn lại không sở hữu class đó vì nó được lấy từ thư viện,... thì bạn không thể có @Inject annotation trong constructor được.
- Mình sẽ đổi tham số truyền vào constructor của MusicPlayer class là MusicDatabase (đây là một abstract class)

```
abstract class MusicDatabase : RoomDatabase() { ... }

class MusicPlayer @Inject constructor(
  private val db: MusicDatabase
) {
  fun play(id: String) { ... }
}
```

- Trong Hilt thì bạn không cần phải lo lắng về các phụ thuộc có tính chất bắc cầu, vì nó sẽ tự động kết nối tất cả các phụ thuộc đó lại với nhau. Tuy nhiên, các bạn cần cho Hilt biết cách cung cấp để tạo các instance đó. Ở ví dụ trên để cung cấp instance của MusicDatabase các bạn hãy sử dụng Hilt modules.
- Hilt modules là một class có @Module annotation , trong class này sẽ có các function cho Hilt biết cách cung cấp để tạo các instance của một số kiểu nhất định. Trong thuật ngữ Hilt, vấn đề này được gọi là bindings .

```
@Module
@InstallIn(SingletonComponent::class)
object DataModule {

  @Provides
  fun provideMusicDB(@ApplicationContext context: Context):
     MusicDatabase {
    return Room.databaseBuilder(
      context, MusicDatabase::class.java, "music.db"
    ).build()
  }
}
```

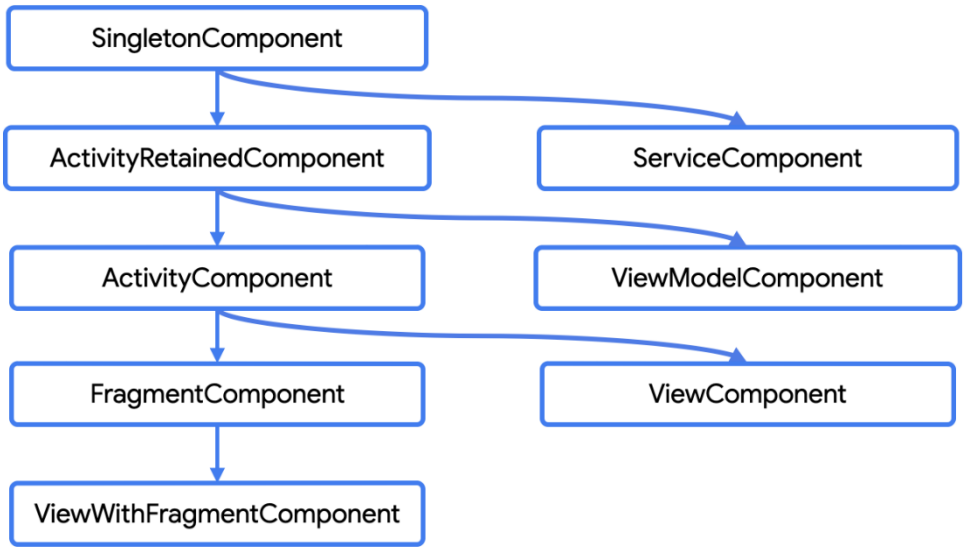
- Function provideMusicDB có @Provides annotation cho Hilt biết cách cung cấp instance có kiểu là MusicDatabase. Phần nội dung bên trong khối block sẽ chứa code mà Hilt cần phải thực thi. Tham số truyền vào cho Hilt biết các phụ thuộc với các kiểu tương ứng, trong trường hợp này là Context, trong Hilt đã có sẵn @ApplicationContext annotation để các bạn có thể lấy được context của ứng dụng rồi. Tóm lại, function này sẽ cung cấp instance của MusicDatabase, và việc khởi tạo instance của MusicPlayer vẫn không có gì thay đổi:

```
@Inject lateinit var player: MusicPlayer
```

- Hilt modules cũng có @InstallIn annotation cho biết thông tin này đã có sẵn trong các vùng chứa hoặc các components phụ thuộc gì (như SingletonComponent ở ví dụ trên), chúng ta sẽ cùng nhau tìm hiểu các components là gì nhé.

Hilt Components

- Hilt Components là một class mà Hilt tạo ra có trách nhiệm cung cấp các instance của từng loại. Tại thời điểm biên dịch, Hilt duyệt qua "đồ thị" phụ thuộc của ứng dụng của bạn và tạo mã để cũng cấp tất cả các kiểu với các phụ thuộc bắc cầu của chúng.
- Hệ thống phân cấp các components của Hilt



Ở ví dụ gần nhất MusicDatabase được binding trong component là SingletonComponent thì nó cũng sẽ có sẵn trong các component còn lại trong hệ thống phân cấp.

- Các component này được tạo tự động tại thời điểm biên dịch, chúng được tạo, quản lý và liên kết với các framework Android tương ứng khi bạn chú thích các class đó bằng @AndroidEntryPoint annotation.
- Các @InstallIn annotation cho module rất hữu ích để kiểm soát những nơi có sự ràng buộc (binding) lại với nhau.

Scoping

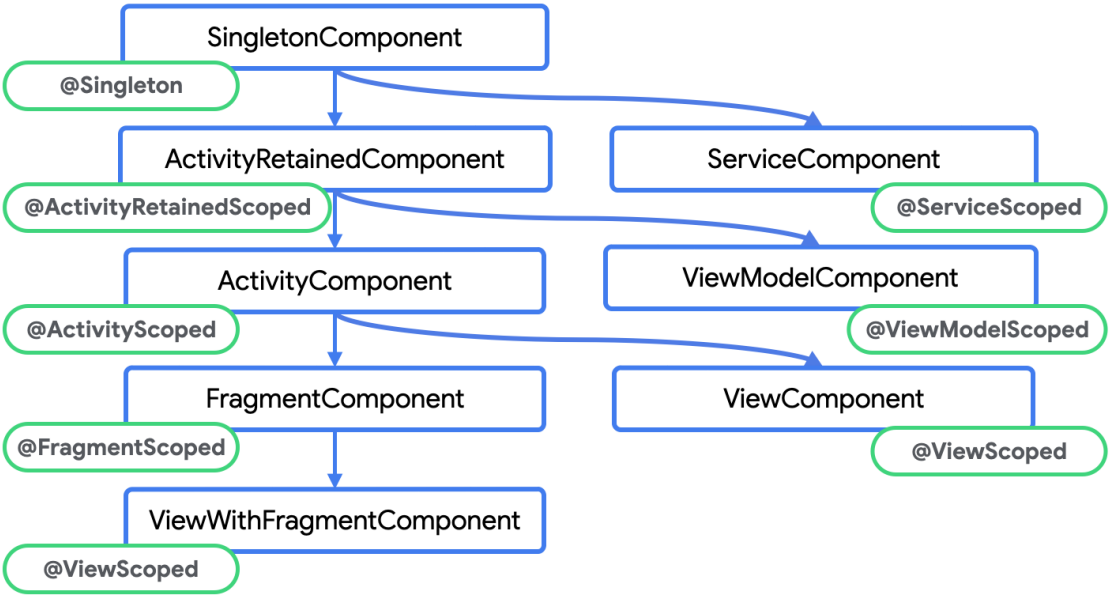
- Ở phần đầu mình có nhắc việc tạo các instance một cách thủ công, nếu bạn để ý thì mỗi lần tạo instance các bạn đang tạo một phiên bản khác của nó. Điều đó hoàn toàn không lý tưởng một chút nào, hãy nên sử dụng chỉ cùng một phiên bản của instance trong toàn bộ ứng dụng. Có rất nhiều cách để có thể thực hiện việc tạo một instance duy nhất, đặc biệt là với ngôn ngữ kotlin, chúng ta sẽ phải viết logic, code thêm rất nhiều đặc biệt với các trường hợp có sự phụ thuộc bắc cầu nhưng với Hilt chỉ cần thêm @Singleton annotation sẽ giải quyết mọi vấn đề.
- Bằng cách sử dụng @Singleton annotation trong các @Provides method, các bạn đang nói với Hilt rằng sẽ luôn chia sẻ cùng một phiên bản của kiểu này trong component đó.

```
@Module
@InstallIn(SingletonComponent::class)
object DataModule {

  @Singleton
  @Provides
  fun provideMusicDB(@ApplicationContext context: Context):
     MusicDatabase {
    return Room.databaseBuilder(
      context, MusicDatabase::class.java, "music.db"
    ).build()
  }
}
```

Ở đoạn code trên Hilt sẽ giúp bạn tạo một instance, một phiên bản duy nhất của MusicDatabase trong toàn bộ ứng dụng.

- @Singleton là một scope annotation. Mỗi Hilt component sẽ có một scope annotation tương ứng



- Nếu bạn muốn xác định một scope trong ActivityComponent hãy sử dụng @ActivityScoped annotation. Các annotation này có thể được sử dụng trong module nhưng chúng cũng có thể sử dụng trong các class mà constructor có @Inject annotation.

Bindings

Có hai loại bindings:

- Unscoped bindings: là các liên kết mà không được chú thích bằng các scope annotaion. Các liên kết này có sẵn cho tất cả các component nếu chúng không được installed trong một module, giống MusicPlayer trong các ví dụ trên.
- Scoped bindings: là các liên kết được chú thích bằng các scope annotaion hoặc các unscoped bindings được installed trong một module của coponent tương ứng, giống MusicDatabase trong các ví dụ trên.

Jetpack Extensions

- Hilt cung cấp tích hợp với các Jetpack library phổ biến nhất như: ViewModel, Navigation, Compose và WorkManager.
- Ngoài ViewModel ra thì khi tích hợp với các Jetpack library khác các bạn sẽ phải thêm một số yêu cầu nhỏ nữa để sử dụng Hilt

```
@HiltViewModel
class ExampleViewModel @Inject constructor(
  private val savedStateHandle: SavedStateHandle,
  private val repository: ExampleRepository
) : ViewModel() {
  ...
}
```

Ngoài việc constructor vẫn có @Inject annotation, để cho Hilt biết cách cung cấp các phiên bản của ViewModel này, bạn chỉ cần chú thích lớp với annotaion là @HiltViewModel. Vậy là xong, hilt sẽ xử lý việc tạo instance của ViewModel này cho bạn, nhưng hãy đảm bảo rằng bạn đã có các liên kết để tạo các instance như SavedStateHandle, ExampleRepository. Bạn muốn lấy instance của ViewModel trên Activity, Fragment,... đơn giản với một dòng code.

```
private val viewModel: ExampleViewModel by viewModels()
```

6.2. Ví dụ và giải thích

Hướng dẫn cài đặt Hilt vào dự án

Bước 1: Thêm Plugin vào build.gradle.kts

```
plugins {
   // Thêm plugin Hilt
   id("com.google.dagger.hilt.android") version "2.51" apply false
}
```

Bước 2: Cấu hình build.gradle.kts

```
plugins {
    id("com.android.application")
    id("kotlin-android")
    id("kotlin-kapt") // Hoặc id("com.google.devtools.ksp") nếu dùng KSP
    id("com.google.dagger.hilt.android")
}

dependencies {
    // Hilt Core
    implementation("com.google.dagger:hilt-android:2.51")
    kapt("com.google.dagger:hilt-compiler:2.51")

    // Hilt tích hợp với Jetpack Compose Navigation
    implementation("androidx.hilt:hilt-navigation-compose:1.2.0")
}
```

Khởi tạo lớp Application

Hilt yêu cầu một lớp Application được đánh dấu để làm "gốc" cho toàn bộ sơ đồ phụ thuộc.

```
@HiltAndroidApp
class TourApplication : Application() {
    // Không cần viết gì thêm ở đây, Hilt sẽ tự động sinh code
}
```

Giải thích: Chú thích @HiltAndroidApp sẽ kích hoạt việc sinh mã nguồn của Hilt, bao gồm một lớp cơ sở cho ứng dụng của bạn, đóng vai trò là thùng chứa phụ thuộc ở cấp độ ứng dụng.

Khai báo Hilt Module

Vì Retrofit hay ApiService là các thư viện bên ngoài (bạn không thể nhảy vào code của họ để sửa), nên bạn cần một cái Module để dạy Hilt cách tạo ra chúng.

```
@Module
@InstallIn(SingletonComponent::class) // Giữ đối tượng này sống suốt vòng đời
của App
object NetworkModule {

    @Provides
    @Singleton // Chỉ tạo 1 lần duy nhất để tiết kiệm RAM
    fun provideRetrofit(): Retrofit {
        return Retrofit.Builder()
            .baseUrl("https://jsonplaceholder.typicode.com/")
            .addConverterFactory(GsonConverterFactory.create())
            .build()
    }

    @Provides
    @Singleton
    fun provideApiService(retrofit: Retrofit): PostApiService {
        return retrofit.create(PostApiService::class.java)
    }
}
```

Giải thích:

- @Module: Đánh dấu đây là nơi cung cấp các "nguyên liệu".
- @InstallIn(SingletonComponent::class): Xác định thời gian sống. Ở đây là Singleton (duy nhất), nghĩa là dù bạn chuyển bao nhiêu màn hình thì cái Retrofit này vẫn là cái cũ, không bị khởi tạo lại.
- @Provides: Nói với Hilt: "Đây là cách tạo ra đối tượng này nè".
- Tham số retrofit: Retrofit trong hàm bên dưới: Hilt cực kỳ thông minh. Nó thấy hàm provideApiService cần một cái Retrofit, nó sẽ tự nhìn lên hàm trên để lấy kết quả trả về và lắp vào.

Tiêm ViewModel

```
@HiltViewModel
class PostViewModel @Inject constructor(
    private val apiService: PostApiService // "Đặt hàng" ở đây
) : ViewModel() {

    private val _posts = MutableStateFlow<List<Post>>(emptyList())
    val posts: StateFlow<List<Post>> = _posts

    fun loadData() {
        viewModelScope.launch {
            val data = apiService.getPosts() // Sử dụng ngay, không cần khởi tạo!
            _posts.value = data
        }
    }
}
```

Giải thích:

- @HiltViewModel: Nói cho Hilt biết lớp này là một ViewModel.
- @Inject constructor: Đây là hành động "đặt hàng". Bạn nói với Hilt: "Khi tạo PostViewModel, hãy nhớ mang theo một cái PostApiService đã chuẩn bị ở Bước 2 cho tôi".
- private val apiService: Bạn không bao giờ phải viết val apiService = Retrofit... nữa. Code cực kỳ sạch và ngắn gọn.

Hiển thị ở Giao diện

```
@AndroidEntryPoint // Bắt buộc phải có ở Activity/Fragment dùng Hilt
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate()
        setContent {
            // Dùng hàm hiltViewModel() thay vì viewModel() thông thường
            val viewModel: PostViewModel = hiltViewModel()

            PostScreen(viewModel)
        }
    }
}
```

Giải thích:

- @AndroidEntryPoint: Đánh dấu đây là "điểm tiếp nhận". Nếu không có nó, Hilt không thể đổ dữ liệu vào Activity này.
- hiltViewModel(): Đây là hàm tiện ích của thư viện androidx.hilt:hilt-navigation-compose. Nó sẽ tự động kết nối với Hilt để lấy về một PostViewModel đã có sẵn apiService bên trong.

6.3. Đồ án

PHẦN 3. XÂY DỰNG ĐỒ ÁN

PHẦN 4. TỔNG KẾT